



Kristu Jayanti College

AUTONOMOUS Bengaluru

Reaccredited 'A++' Grade by NAAC | Affiliated to Bengaluru North University

PROJECT REPORT ON

“Blockchain-Based University Certificate Issuance and Verification System”

Submitted in partial fulfillment of the requirements for Enterprise
Computing Project Lab for 3rd Semester

Master of Computer Applications (MCA)

Submitted by

Mr. Alwin James 24MCAB08

Mr. Rohan Benny 24MCAB48

Under the guidance of

Dr. Princess Maria John



Kristu Jayanti College

AUTONOMOUS **Bengaluru**

Reaccredited 'A++' Grade by NAAC | Affiliated to Bengaluru North University

CERTIFICATE

This is to certify that the project titled “***Blockchain-Based University Certificate Issuance & Verification System***” has been satisfactorily completed by **Mr. Alwin James** with **24MCAB08**, in partial fulfillment of the requirements for ***Enterprise Computing Project*** with course code **MCC2P2B31**, for the 3rd Semester MCA course during the academic semester from July 2025 to November 2025 as prescribed by Bangalore North University.

Faculty In-charge

Head of the Department

Valued by

Examiner 1: _____

Date :

Examiner 2: _____

Centre: Kristu Jayanti College

ACKNOWLEDGEMENT

First of all, we would like to thank the God Almighty for all the blessings he has showered on us. Our spiritual quotient gave us more strength and motivation that helped immensely.

We would like to thank **Rev. Fr. Dr. Augustine George**, our Principal, for providing us his constant guidance and support. I would also like to thank **Rev. Fr. Dr. Lijo P Thomas**, our Vice Principal, for providing us with the best facilities.

We are extremely thankful to our Head of the Department **Dr. Muruganantham A** for his enormous guidance and motivation and also giving us the essential support in the form of allocating comfortable lab hours and necessary resources.

We would like to extend our heartfelt thanks to **Dr. Princess Maria John**, our project guide for providing us the necessary details related to project development and process identification enabling us to finish the project within the stipulated time.

We thank all other faculty members who helped us a lot in completing this project. The computer lab was always open for us. We thank the lab administrator and other technical staff for their help and support.

We thank our class mates, who have pointed out errors and guided us a lot and we thank each and every one who has helped us.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Abstract	vi
	List of Figures	vii
	List of Tables	viii
	INTRODUCTION	
1	1.1 Problem Statement	1
	1.2 Project Description (includes Module Description)	1
	SYSTEM STUDY	
2	2.1 Existing System	4
	2.2 Proposed System	5
	2.3 Use Case Diagram	6
	2.4 Data Flow Diagram	6
	2.5 Activity Diagram	8
	SYSTEM CONFIGURATION	
3	3.1 Hardware Configuration	9
	3.2 Software Configuration	10
	SOFTWARE DESCRIPTION	
4	4.1 Overview Front End	11
	4.2 Overview Back End	13
	SYSTEM DESIGN	
5	5.1 Architectural	15
	5.2 Input Design	18
	5.3 Output Design	20
	5.4 Database Design	22
6	SOURCE CODE	26
7	SYSTEM TESTING	54
8	SYSTEM IMPLEMENTATION / DEPLOYMENT	58

CHAPTER NO	TITLE	PAGE NO
9	CONCLUSION AND FUTURE ENHANCEMENT	61
10	APPENDICES	
	10.1 Screenshots	63
11	REFERENCES	67

ABSTRACT

The Blockchain-Based University Certificate Issuance and Verification System is a decentralized web application that provides a secure and transparent method for issuing and validating academic certificates. The project uses blockchain technology and IPFS storage to ensure data integrity and eliminate certificate forgery.

Universities can upload certificate templates in HTML/EJS format with predefined placeholders and generate certificates by filling out simple forms. Each issued certificate is converted to a PDF, stored on IPFS, and recorded on the Ethereum blockchain with a unique certificate ID. This guarantees authenticity, immutability, and long-term accessibility.

Students can log in using their email to view and download their issued certificates. Even if a student registers after issuance, the system automatically maps certificates to their email. A public verification portal allows employers or verifiers to confirm certificate authenticity using the blockchain ID, displaying both metadata and a PDF preview.

The system is developed using React.js, Node.js, Express.js, MongoDB, Solidity, and Ether.js. By integrating decentralized storage and blockchain, the project eliminates manual verification delays, prevents document tampering, and establishes a secure, efficient framework for digital certificate management in educational institutions.

LIST OF FIGURES

Figure No	Title	Page No
2.1	Use Case	6
2.2	DFD Level 0	6
2.3	DFD Level 1	7
2.4	Activity Diagram	8
5.1	Architectural Design	15
5.2	Login Input Design	18
5.3	Upload Certificate Input Design	18
5.4	Issue Certificate Input Design	19
5.5	Create Account Input Design	19
5.6	Login Output Design	20
5.7	Upload Certificate Output Design	20
5.8	Issue Certificate Output Design	21
5.9	Create Account Output Design	21
10.1	Landing Page	63
10.2	Login Page	63
10.3	Admin Dashboard	64
10.4	University Dashboard	64
10.5	Student Dashboard	65
10.6	Certificate Verification	66
10.7	Issued Certificates	66

LIST OF TABLES

Table No	Title	Page No
3.1	Hardware Configuration	9
3.2	Software Configuration	10
4.1	Front-End Technology	12
4.2	Back-End Technology	14
5.1	Users Table	22
5.2	Pending Requests Table	22
5.3	Templates Table	23
5.4	Certificates Table	24
7.1	Admin Module Test Cases	54
7.2	University Module Test Cases	55
7.3	Student Module Test Cases	56
7.4	Verifier Module Test Cases	57

1. INTRODUCTION

1.1 Problem Statement

Traditional methods of issuing and verifying academic certificates are time-consuming, prone to human error, and vulnerable to forgery or unauthorized alteration. Educational institutions often rely on paper-based or centralized digital systems that lack transparency and security, making it difficult for employers and verifiers to confirm the authenticity of certificates.

In many cases, manual verification requires direct communication with universities, causing significant delays and administrative burden. Furthermore, students risk losing their certificates due to physical damage or data loss in centralized systems.

Therefore, there is a need for a secure, transparent, and tamper-proof digital platform that allows universities to issue certificates reliably, students to access them easily, and employers or verifiers to instantly validate their authenticity without intermediaries.

The Blockchain-Based University Certificate Issuance and Verification System addresses this problem by leveraging blockchain technology and decentralized storage (IPFS) to ensure data integrity, eliminate forgery, and simplify the verification process.

1.2 Project Description

The Blockchain-Based University Certificate Issuance and Verification System is a decentralized web application developed to revolutionize the way educational institutions issue, manage, and verify academic certificates. Traditional methods of certificate management often rely on paper-based processes or centralized databases, which are prone to forgery, human error, and delays in verification. This project integrates Blockchain technology and IPFS (InterPlanetary File System) to create a secure, transparent, and tamper-proof framework for digital certificates.

The system includes four major user roles—Admin, University, Student, and Public Verifier—each performing distinct functions to maintain the integrity and efficiency of the overall system. The Admin monitors the platform and approves universities, the Universities issue digital certificates through customizable templates, the Students can access and download their verified credentials, and the Public Verifiers can instantly validate certificates using the blockchain ID. Once a certificate is issued, its PDF version is uploaded to IPFS, and its

metadata, including the IPFS hash, issue date, and university ID, is stored on the Ethereum blockchain through a smart contract developed in Solidity.

The system is built using a modern web stack: React.js for the frontend, Node.js and Express.js for backend services, MongoDB for database management, and Ether.js for blockchain interactions. The certificates are permanently accessible, verifiable, and immune to tampering. By leveraging decentralized technology, the system ensures data transparency, eliminates manual verification delays, and offers a scalable and secure platform for global academic verification.

1.2.1 Admin Module

The Admin Module acts as the supervisory layer of the system, ensuring smooth operation and maintaining institutional integrity. The admin is responsible for managing university accounts, approving new university registrations, and ensuring that only verified institutions gain access to certificate issuance privileges. This module helps maintain the trustworthiness of the system by preventing unauthorized universities from issuing certificates. Additionally, the admin oversees the performance of universities and ensures compliance with system protocols. This module plays a key role in sustaining platform transparency and overall governance.

1.2.2 University Module

The University Module is one of the core functional components of the system, allowing universities to manage certificate issuance efficiently. After approval from the admin, universities can log in to their respective dashboards and perform multiple operations such as uploading certificate templates, adding course lists, and issuing certificates. Templates are uploaded in HTML or EJS format with predefined placeholders such as `{{student_name}}`, `{{course}}`, and `{{issue_date}}`. During issuance, the university fills these placeholders through a web form, and the system automatically generates a PDF certificate with a unique blockchain ID embedded in it. The generated certificate is then uploaded to IPFS, and its metadata—comprising the IPFS hash, issue date, and university ID—is recorded on the Ethereum blockchain using a Solidity smart contract. This process guarantees that certificates are tamper-proof, verifiable, and permanently accessible.

1.2.3 Student Module

The Student Module provides students with a secure and personalized dashboard where they can view and download all certificates linked to their registered email address. Each certificate

includes details such as the blockchain ID, certificate type, IPFS link, and issue date. Even if a student is not registered at the time of issuance, the system stores the certificate under their email ID, ensuring that it appears automatically once they sign up. The student module ensures lifelong access to certificates through decentralized storage, removing the dependency on university servers and enabling students to easily share or verify their credentials with employers and institutions.

1.2.4 Verification Module (Public Portal)

The Verification Module serves as a publicly accessible portal for certificate verification. It allows any verifier—such as an employer, institution, or third-party organization—to check the authenticity of a certificate by entering its blockchain ID. The system retrieves certificate metadata from the blockchain and the corresponding file from IPFS, displaying both the certificate details and a PDF preview. If the certificate does not exist or has been altered, the system clearly displays an “Invalid Certificate” message. This module enhances transparency, reduces manual verification delays, and ensures that only valid certificates are recognized and trusted globally.

1.2.5 Blockchain and IPFS Storage Module

The Blockchain and IPFS Storage Module forms the backbone of the entire system, combining decentralization, immutability, and permanent accessibility. Whenever a certificate is issued, the system uploads the generated PDF to IPFS through the Pinata API, which returns a unique hash known as a Content Identifier (CID). This hash serves as the permanent address of the certificate on IPFS. Simultaneously, certificate metadata—including the certificate ID, university ID, student email, issue date, and IPFS hash—is recorded on the Ethereum blockchain using a smart contract. The blockchain ensures data integrity and prevents unauthorized modifications, while IPFS guarantees decentralized storage of files. This dual approach provides a trustless, transparent, and tamper-proof verification mechanism that ensures certificates remain accessible forever, regardless of server failures or institutional changes.

2. SYSTEM STUDY

2.1 Existing System

In the current academic landscape, most universities and educational institutions rely on traditional manual processes or centralized digital systems for issuing and verifying academic certificates. Although these systems have served institutions for decades, they face significant challenges related to security, authenticity, efficiency, and accessibility.

Traditionally, certificates such as degree certificates, marks cards, and transfer certificates are issued in paper-based formats or generated using conventional digital tools like word processors and spreadsheet applications. These documents are often printed and physically distributed to students or sent via email as digital copies. However, such certificates are vulnerable to duplication, alteration, and forgery, since they lack a robust mechanism to prove authenticity. With the increasing availability of graphic editing software and online design tools, counterfeit certificates can be created with minimal effort, making it extremely difficult for employers or other educational institutions to differentiate genuine credentials from fake ones.

When verification is required—such as during job recruitment, higher education admissions, or international migration—the process is largely manual and time-consuming. Verifiers are required to contact the issuing university or board directly, either by email, post, or through official communication channels, to confirm the legitimacy of the certificate. This verification process may take days or even weeks, especially when dealing with cross-border verification. Moreover, due to administrative workload and human dependency, there is always a possibility of errors or fraudulent confirmations, which further reduces trust in the system.

In many universities, even digital certificate management relies on centralized databases or local servers, where all issued certificates and student data are stored. This centralized nature poses serious data security and integrity risks. If the database is hacked, corrupted, or accidentally deleted, valuable student records may be lost or compromised. Recovery and reissuance of certificates in such scenarios are often complex, expensive, and time-intensive. Additionally, these centralized systems are susceptible to unauthorized modifications, where malicious actors can manipulate data for personal gain, undermining the credibility of the institution.

Another major drawback of the existing system is the lack of transparency and accessibility. There is no standardized or universal platform where third parties, such as employers or institutions, can instantly verify a certificate's authenticity. As a result, verifiers are dependent on the university's internal processes, which vary from one institution to another. This lack of interoperability creates administrative delays and inefficiencies, affecting both the verification process and institutional reputation.

Furthermore, students often face challenges in maintaining lifelong access to their certificates. Physical copies are prone to damage or loss, and digital copies stored locally can be deleted or misplaced. In such cases, students are required to apply for duplicate certificates, which involves additional paperwork, fees, and processing time.

2.2 Proposed System

The Blockchain-Based University Certificate Issuance and Verification System is proposed to overcome the limitations of traditional and centralized certificate management methods by using blockchain technology and decentralized storage. This system provides a secure, transparent, and tamper-proof solution for issuing, storing, and verifying academic certificates.

In the proposed system, each university can create and upload customizable certificate templates in HTML/EJS format with predefined placeholders for student details. When a certificate is issued, the university fills in the placeholder values through an issuance form, and the system automatically generates a PDF certificate. This certificate is then uploaded to the InterPlanetary File System (IPFS), ensuring decentralized and permanent storage.

Simultaneously, essential certificate metadata—such as the certificate ID, IPFS hash, issue date, and university ID—is recorded on the Ethereum blockchain through a smart contract. This blockchain record acts as a digital proof of authenticity and guarantees that the data cannot be altered or deleted by any unauthorized party.

Students can log in using their registered email IDs to access and download their issued certificates. If a student registers after the certificate has been issued, the system automatically links the stored certificate to their email address, ensuring seamless access. A Public Verification Portal allows anyone, such as employers or educational institutions, to verify a certificate's authenticity by entering its blockchain ID.

By leveraging the immutability of blockchain and distributed nature of IPFS, the proposed system ensures data transparency, prevents certificate forgery, and eliminates the need for

manual verification. It simplifies certificate management for universities, provides students with lifelong access to their credentials, and enables instant, trustworthy verification for third parties worldwide.

2.3 Use Case Diagram

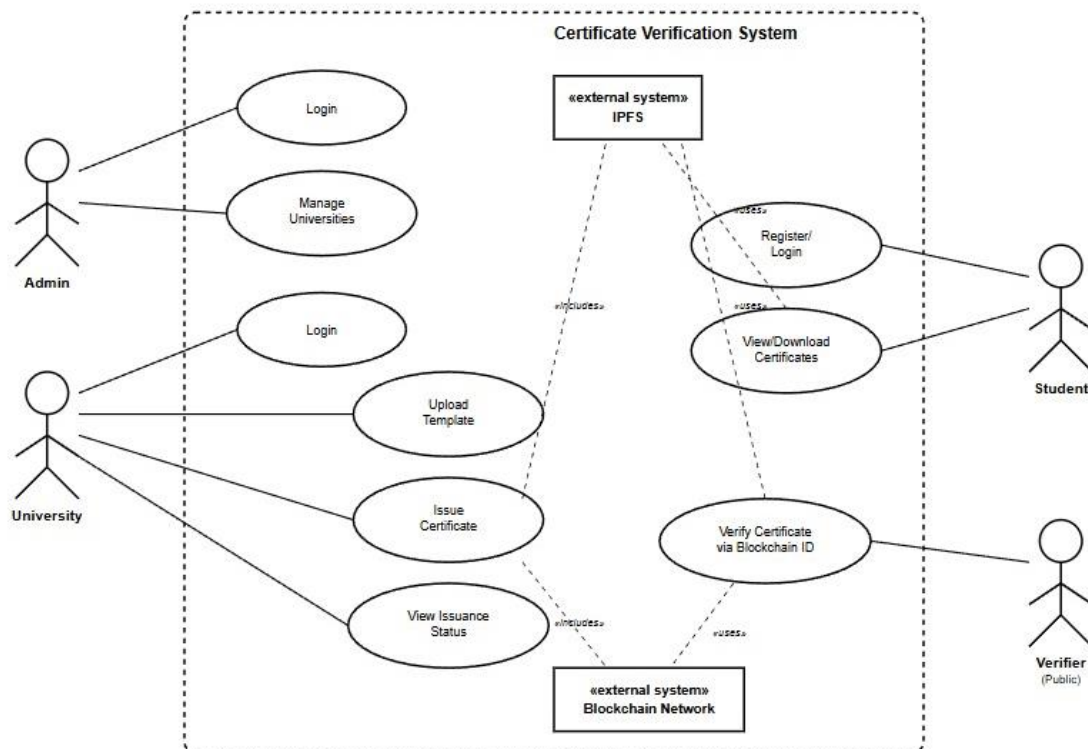


Figure 2.1 Use Case Diagram

2.4 Data Flow Diagram

DFD Level 0:

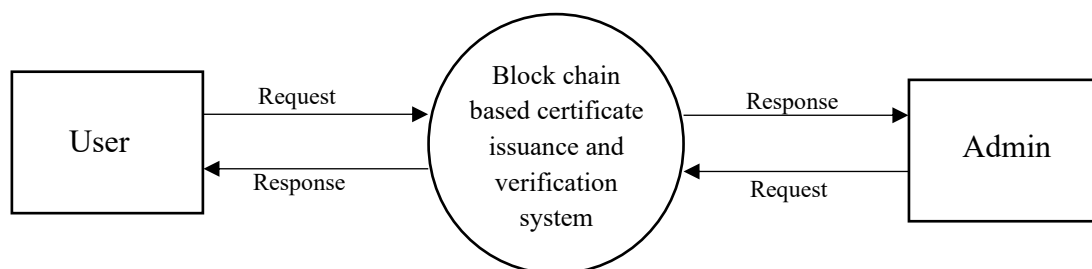
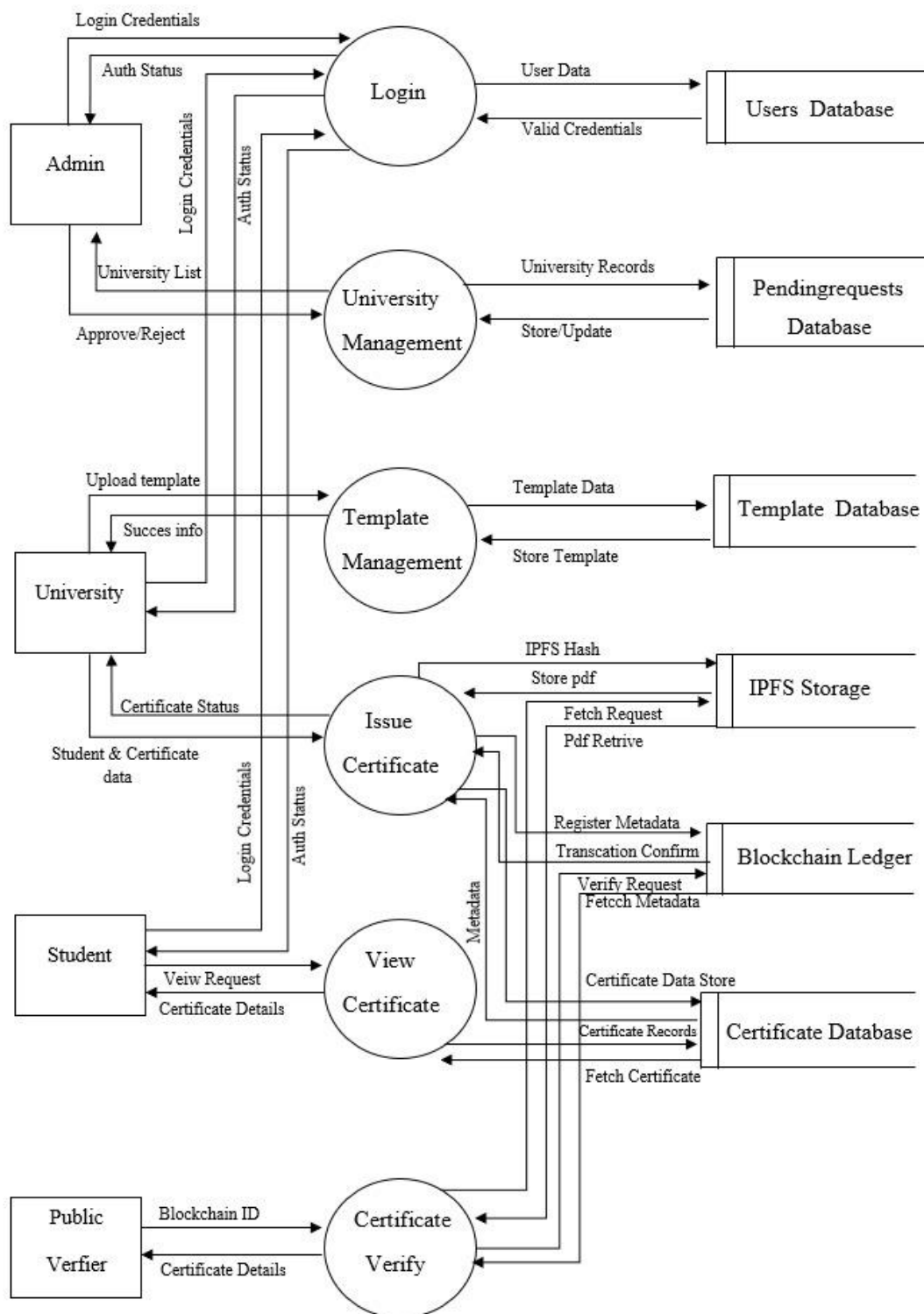


Figure 2.2 DFD Level 0

DFD Level 1:*Figure 2.3 DFD Level 1*

2.5 Activity Diagram

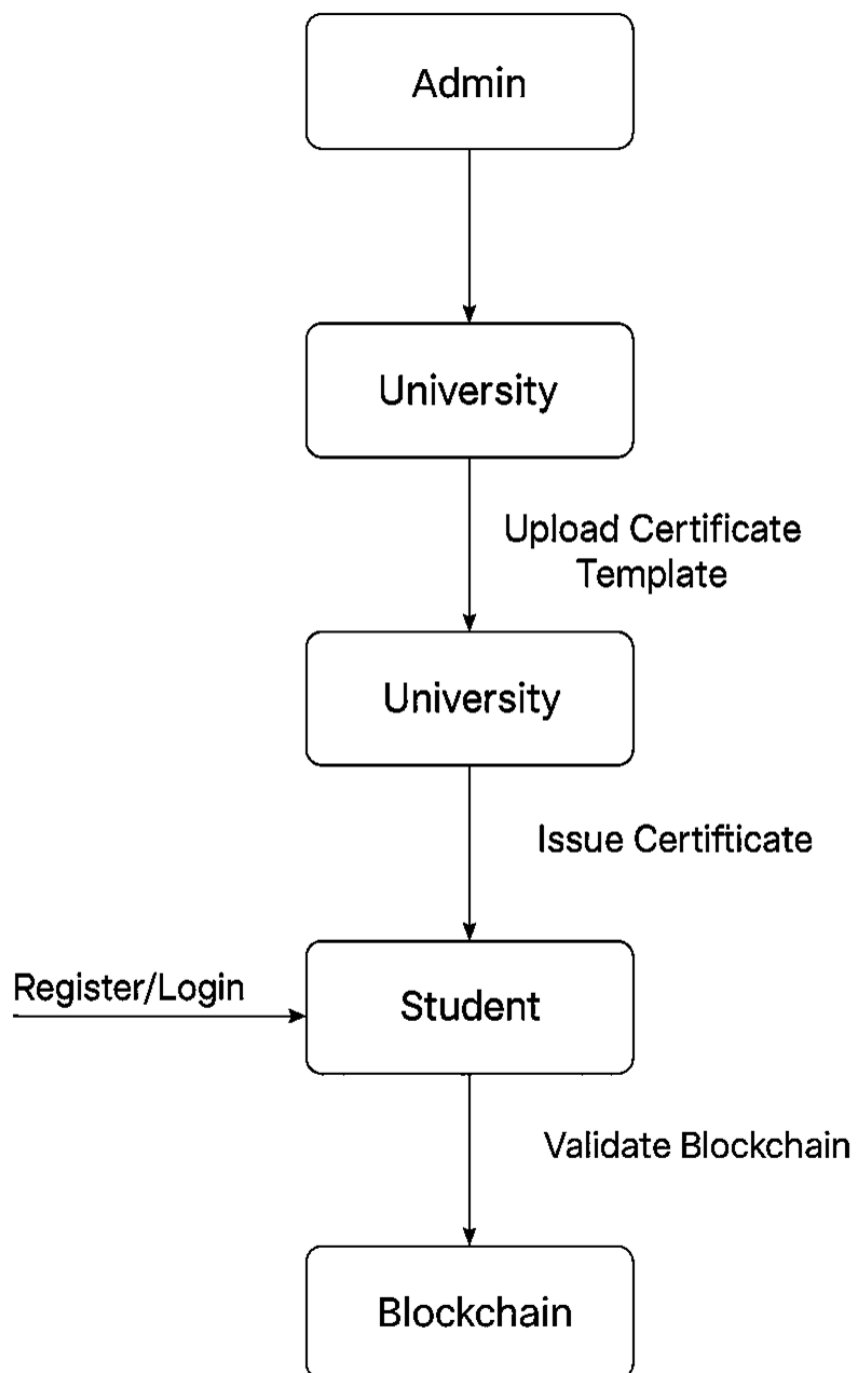


Figure 2.4 Activity Diagram

3. SYSTEM CONFIGURATION

3.1 Hardware Configuration

The Blockchain-Based University Certificate Issuance and Verification System is a web-based decentralized application that requires a moderate hardware setup for both development and deployment environments. Since the system integrates blockchain interaction, IPFS file storage, and a web interface, the configuration must ensure sufficient processing capability, memory, and storage to handle certificate generation, blockchain transactions, and file uploads efficiently.

Table 3.1 Hardware Configuration:

Component	Specification / Description
Processor	Intel Core i5 or equivalent (Minimum 2.5 GHz)
RAM	Minimum 8 GB
Hard Disk	Minimum 500 GB HDD or 256 GB SSD
Graphics Card	Integrated Graphics / NVIDIA GeForce GTX 1050 or higher
Monitor	14" or larger display with 1080p resolution
Input Devices	Standard Keyboard, Mouse / Touch Input
Audio Devices	Speakers / Headphones with microphone support
Network	Stable high-speed Internet connection
Operating System	Windows 10 / 11, macOS, or Linux
Support	Compatible with local and cloud-based environments (supports blockchain and IPFS connectivity)

This configuration ensures stable and efficient performance during both the development and deployment stages of the project. It provides sufficient computing power for smart contract execution, PDF generation, blockchain transactions, and IPFS interactions. The recommended setup enables seamless communication between the client and server modules, ensuring smooth operation, reliable connectivity, and overall system scalability.

3.2 Software Configuration

The Blockchain-Based University Certificate Issuance and Verification System is developed using a combination of modern web technologies, blockchain frameworks, and decentralized storage solutions. The software configuration ensures seamless integration between the frontend, backend, blockchain, and IPFS layers, enabling a secure, scalable, and user-friendly platform for certificate generation, storage, and verification.

Table 3.2 Software Configuration:

Component	Specification / Description
Operating System	Windows 10 / 11, Linux (Ubuntu 20.04 LTS or higher), macOS
Frontend Technologies	React.js, HTML5, CSS3, JavaScript, Tailwind CSS
Backend Technologies	Node.js, Express.js
Database	MongoDB (Local or Cloud via MongoDB Atlas)
Blockchain Platform	Ethereum Network (Testnet / Mainnet)
Smart Contract Language	Solidity
Blockchain Development Framework	Hardhat
Blockchain Interaction Library	Ethers.js
Decentralized Storage	IPFS (InterPlanetary File System)
IDE / Code Editor	Visual Studio Code
Browser Support	Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)
Package Manager	npm (Node Package Manager)
PDF Generation Library	html-pdf / Puppeteer

4. SOFTWARE DESCRIPTION

4.1 Overview Front End

The front-end of the Blockchain-Based University Certificate Issuance and Verification System serves as the primary interface that connects users to the application's underlying blockchain and database functionalities. It is designed to offer a clean, intuitive, and interactive environment for all user types — including Admins, Universities, Students, and Public Verifiers. The interface focuses on accessibility, simplicity, and usability while maintaining high performance across various devices and browsers.

Built using React.js, the front-end follows a component-based architecture that allows modular development and code reusability. Each interface component — such as login pages, dashboards, certificate upload forms, and verification portals — is designed as an independent module that interacts with backend APIs to fetch, display, and update data dynamically. The system ensures that all real-time certificate issuance, viewing, and verification operations occur seamlessly without page reloads, improving the overall user experience.

The design emphasizes responsiveness and security. All user actions, such as login authentication, template uploads, and verification requests, are securely transmitted to the backend through encrypted HTTP requests. A consistent visual layout is maintained across different roles using clean CSS styling and structured UI elements. The integration of QR codes for certificate blockchain IDs ensures quick accessibility for verification purposes, making the system both user-friendly and technically robust.

Key Features

➤ Role-Based User Interfaces

Separate, customized dashboards for Admins, Universities, Students, and Public Verifiers, ensuring clarity and restricted access based on roles.

➤ **Dynamic Certificate Rendering**

Certificates are generated and displayed dynamically using placeholders within uploaded HTML/EJS templates, ensuring flexibility in design.

➤ **Responsive Design and Navigation**

Optimized for all screen sizes with a smooth navigation flow powered by React Router, enabling instant transitions between system pages.

➤ **Secure Authentication and Session Management**

Login sessions are maintained using JSON Web Tokens (JWT), ensuring that data and user credentials remain protected throughout interaction

Technologies Used in Front-End

Table 4.1 Front-End Technology:

Technology	Purpose
React.js	Core framework for building the dynamic and modular user interface.
HTML5 & CSS3	Used for structuring and styling responsive and accessible web pages.
JavaScript	Provides logic, event handling, and interactivity for UI components.
Axios	Handles secure HTTP requests and data exchange with backend APIs.
React Router	Enables client-side routing for smooth page transitions without reloads.
JWT (JSON Web Token)	Ensures secure user authentication and session management.

This front-end setup ensures that the system remains lightweight, responsive, and secure, delivering a seamless and interactive experience for all users while maintaining strong connectivity with the blockchain and backend layers.

4.2 Overview Back End

The back-end of the Blockchain-Based University Certificate Issuance and Verification System serves as the core logic layer that manages data processing, blockchain interactions, and communication between the front-end interface and decentralized components. It ensures that all user actions — such as certificate issuance, verification, and storage — are executed securely and efficiently while maintaining system integrity and data consistency.

Developed using Node.js and Express.js, the back-end provides a robust RESTful API architecture that connects to both the MongoDB database and the Ethereum blockchain network. Each API endpoint handles specific tasks such as user authentication, template management, certificate generation, and verification. The back-end also integrates with the IPFS (InterPlanetary File System) for decentralized storage of certificates, ensuring that all academic documents remain immutable and permanently accessible.

Security and reliability are at the heart of the back-end design. All data transmissions occur over secure protocols, and sensitive operations like certificate issuance and blockchain registration are cryptographically verified. Additionally, middleware functions are implemented to validate user roles, handle authentication using JSON Web Tokens (JWT), and manage error responses gracefully. This architecture ensures scalability, modularity, and fault tolerance, making the system dependable for both academic institutions and verifiers.

Key Features

➤ RESTful API Architecture

The backend provides well-structured APIs that connect the frontend, database, blockchain, and IPFS layers seamlessly, ensuring modular and scalable system operations.

➤ Blockchain Integration

Smart contracts deployed on the Ethereum network record and verify certificate metadata, ensuring immutability and transparency of issued certificates.

➤ IPFS Integration for Decentralized Storage

Generated certificate files (PDFs) are uploaded to IPFS through the Pinata API, and their hashes are stored on the blockchain for secure and permanent access.

➤ **Authentication and Role Management**

The back-end implements JWT-based authentication to secure access to routes and assign role-specific permissions to Admins, Universities, and Students.

➤ **Database Management and Data Linking**

MongoDB is used to manage and link data across users, universities, templates, and certificates, ensuring efficient query handling and data retrieval.

Technologies Used in Back-End

Table 4.2 Back-End technologies:

Technology	Purpose / Description
Node.js	Provides a fast and scalable runtime environment for server-side development.
Express.js	Framework for building RESTful APIs and managing backend routing and middleware.
MongoDB	NoSQL database for storing user, template, and certificate metadata efficiently.
Solidity	Programming language for developing smart contracts on the Ethereum blockchain.
Hardhat	Development environment for compiling, testing, and deploying smart contracts.
Ethers.js	Library for interacting with the Ethereum blockchain and smart contracts
IPFS (Pinata API)	Used for decentralized storage of certificate PDFs and returning IPFS hashes.

5. SYSTEM DESIGN

5.1 Architectural

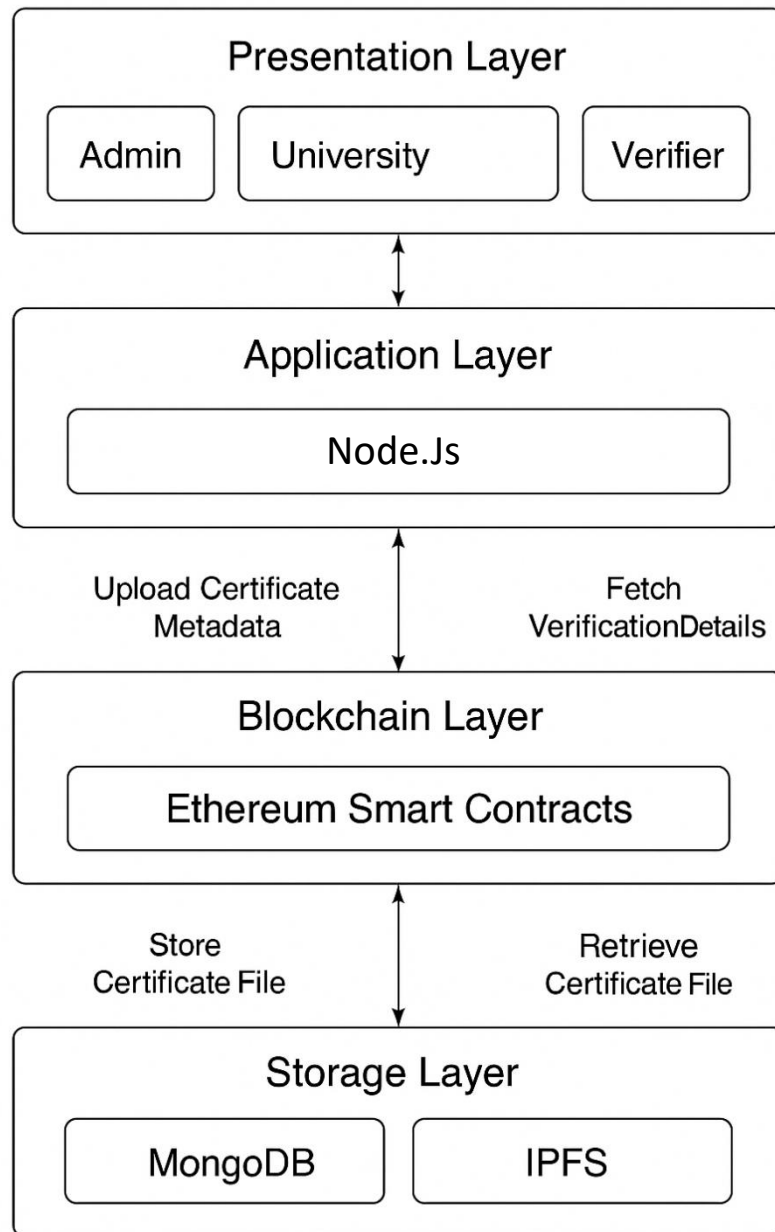


Figure 5.1 Architectural Diagram

The architecture of the *Blockchain-Based University Certificate Issuance and Verification System* is designed using a layered approach to ensure security, scalability, and transparency in academic credential management. The system integrates modern web technologies with blockchain and decentralized storage to eliminate fraud and streamline verification. The architecture is divided into four key layers — Presentation Layer, Application Layer, Blockchain Layer, and Storage Layer, each responsible for distinct functionalities that collectively enable the secure issuance and verification of certificates (refer fig. 5.1).

1. Presentation Layer

The Presentation Layer represents the front-end interface through which all users interact with the system. It includes four main user roles — Admin, University, Student, and Verifier — each having access to specific functionalities based on their privileges.

- Admin can manage and approve university registrations.
 - University can upload certificate templates, issue certificates, and manage student data.
 - Students can view and download blockchain-verified certificates.
 - Verifiers can validate the authenticity of certificates using blockchain IDs.
- This layer is built using React.js, ensuring a responsive, user-friendly, and interactive web experience.

2. Application Layer

The Application Layer acts as the core logic layer and handles communication between the frontend, blockchain, and database. It is developed using Node.js and Express.js APIs, which manage all server-side operations. This layer processes user requests such as login, template uploads, certificate issuance, and verification. It ensures secure data flow by validating inputs, generating dynamic PDFs, uploading them to IPFS, and interacting with smart contracts. The data flow is bidirectional — certificate data is sent to the blockchain for recording and later retrieved for verification, ensuring seamless connectivity between components.

3. Blockchain Layer

The Blockchain Layer is the backbone of the system that ensures trust, transparency, and immutability. It uses Ethereum Smart Contracts to store certificate metadata such as the certificate ID, university ID, IPFS hash, and issue date. When a university issues a certificate, the metadata is uploaded to the blockchain, creating a tamper-proof record. During verification,

the blockchain retrieves these details to confirm authenticity. The smart contracts serve as a decentralized ledger, eliminating manual validation and preventing data manipulation or forgery.

4. Storage Layer

The Storage Layer is responsible for managing both decentralized and centralized data storage. It includes IPFS (InterPlanetary File System) and MongoDB.

- IPFS securely stores the actual certificate files (PDFs) in a decentralized manner, ensuring permanent accessibility and eliminating single points of failure.
- MongoDB is used for storing user credentials, templates, and linkage information between students, universities, and certificates. Both systems interact with the Application Layer through two-way communication: data is uploaded during issuance and retrieved during verification.

5.2 Input Design

This is the user login interface for the Blockchain-Based Certificate Issuance and Verification System, allowing users to securely sign in using their registered email and password. It provides a clean design with options for password recovery and new user registration (refer fig 5.2).

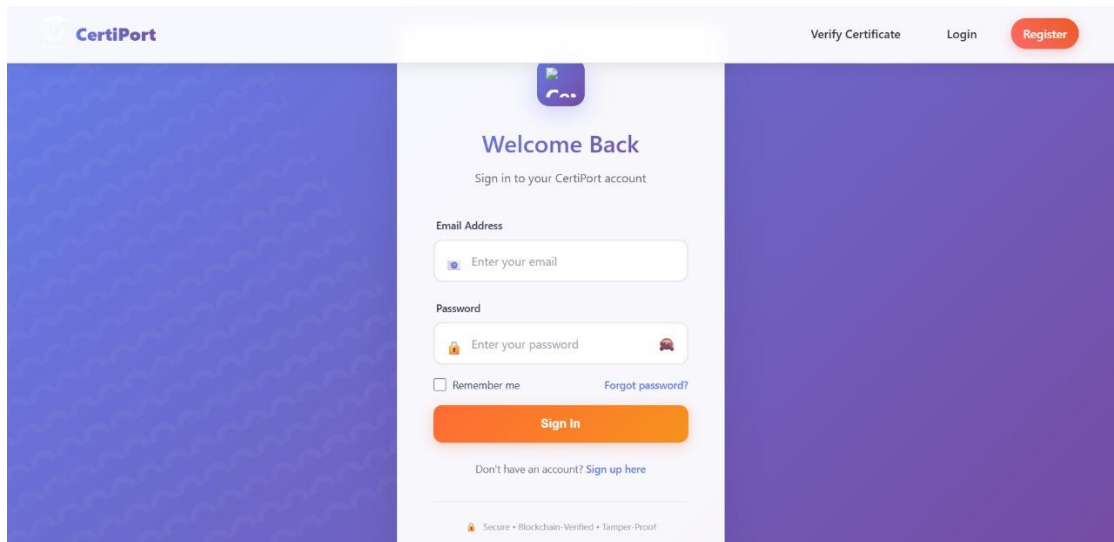


Figure 5.2 Login Input Design

This template upload interface enables universities to upload customizable certificate templates in .ejs or .html format. It allows administrators to define certificate types and manage reusable templates for issuing blockchain-verified academic certificates (refer fig 5.3).

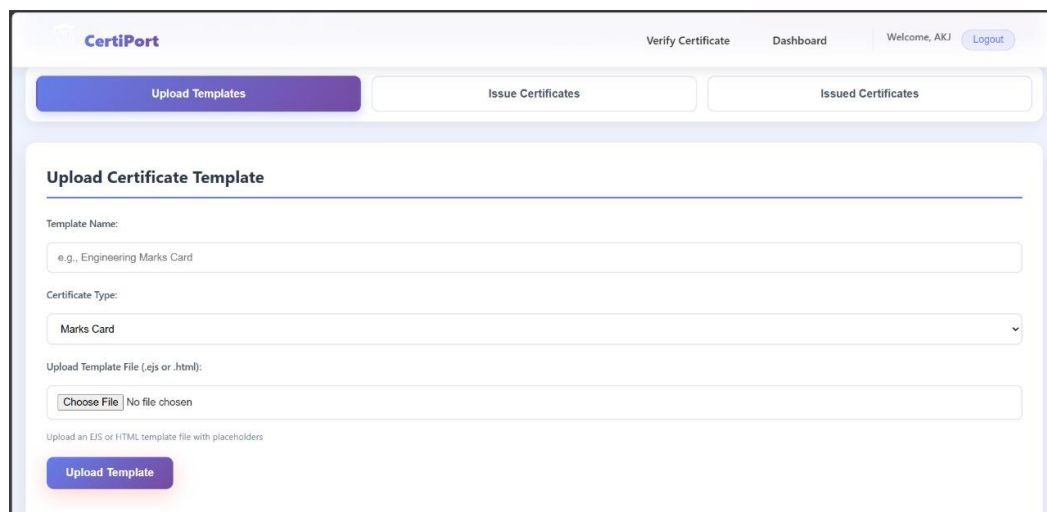
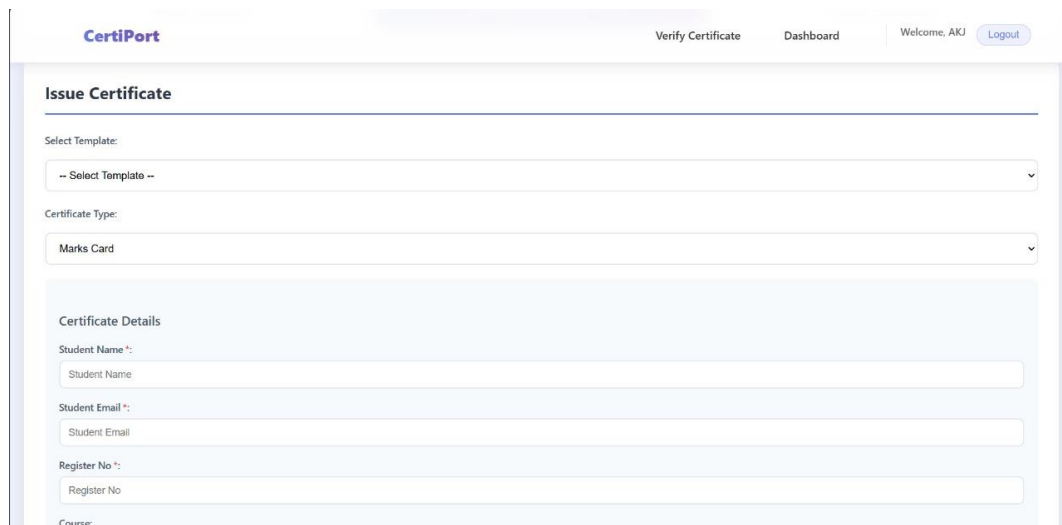


Figure 5.3 Upload Certificate Input Design

This certificate issuance interface allows universities to select a pre-uploaded template and fill in student details such as name, email, register number, and course. It streamlines the process of generating blockchain-verified certificates through structured input form (refer fig. 5.4).

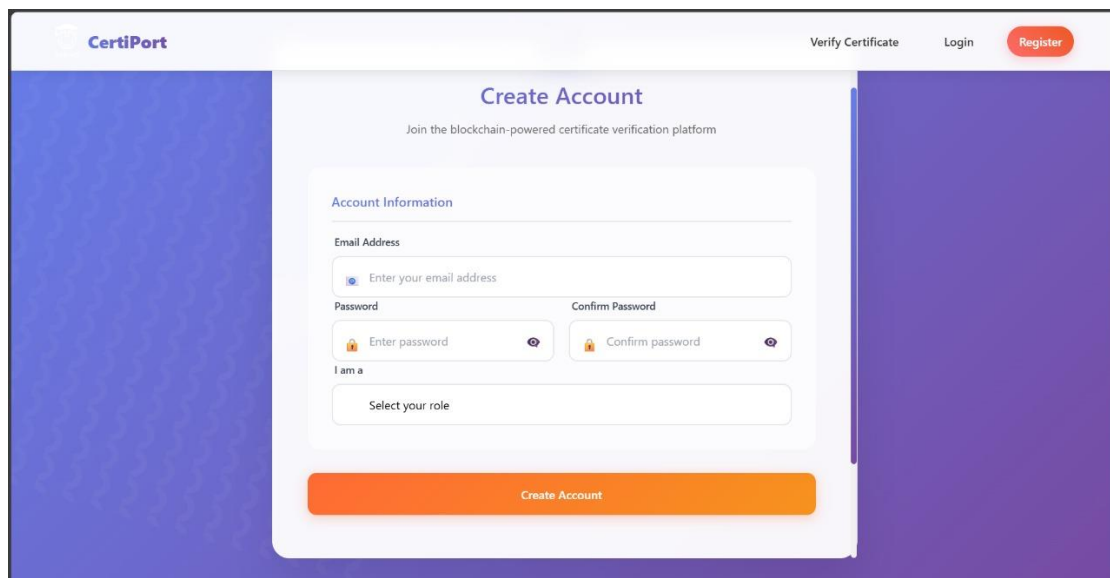


The screenshot shows the 'Issue Certificate' interface on the CertiPort website. The header includes the CertiPort logo, navigation links for 'Verify Certificate' and 'Dashboard', and a user greeting 'Welcome, AKJ' with a 'Logout' button. The main form is titled 'Issue Certificate' and contains the following sections:

- Select Template:** A dropdown menu with the placeholder text '-- Select Template --'.
- Certificate Type:** A dropdown menu with 'Marks Card' selected.
- Certificate Details:** A section with four input fields:
 - Student Name *:** Labeled 'Student Name'.
 - Student Email *:** Labeled 'Student Email'.
 - Register No *:** Labeled 'Register No'.
 - Course:** An unlabeled input field.

Figure 5.4 Issue Certificate Input Design

This account creation interface enables new users such as universities, students, or verifiers to register on the blockchain-based certification platform. It captures essential credentials including email, password, and user role, ensuring secure authentication and access to role-specific functionalities.



The screenshot shows the 'Create Account' interface on the CertiPort website. The header includes the CertiPort logo, navigation links for 'Verify Certificate' and 'Login', and a red 'Register' button. The main form is titled 'Create Account' with the subtitle 'Join the blockchain-powered certificate verification platform'. The form is divided into two vertical panels. The left panel, titled 'Account Information', contains the following fields:

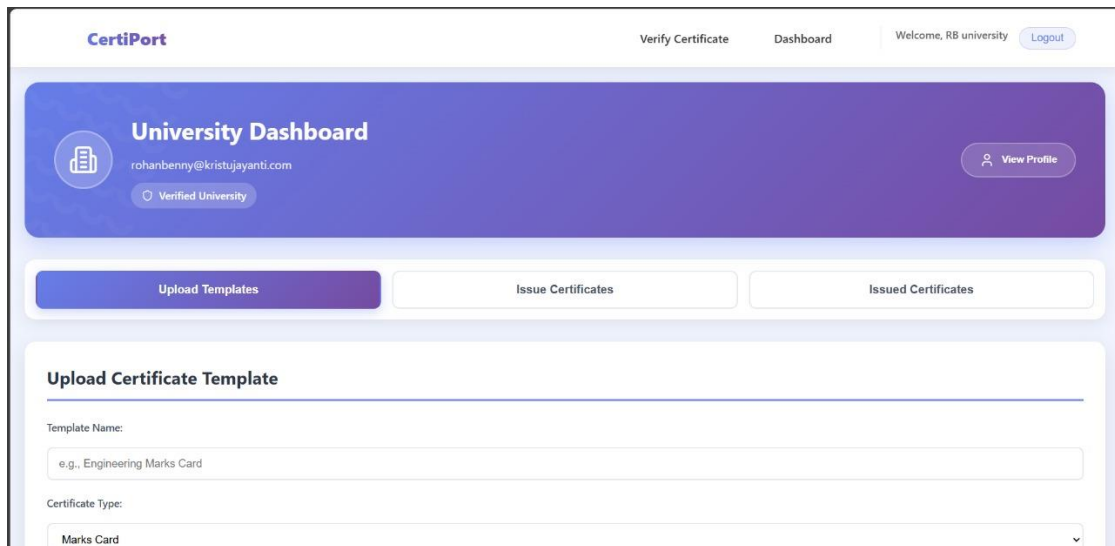
- Email Address:** Labeled 'Enter your email address'.
- Password:** Labeled 'Enter password' with a password strength indicator.
- Confirm Password:** Labeled 'Confirm password' with a password strength indicator.
- I am a:** Labeled 'Select your role'.

A large orange 'Create Account' button is positioned at the bottom of the form.

Figure 5.5 Create Account Input Design

5.3 Output Design

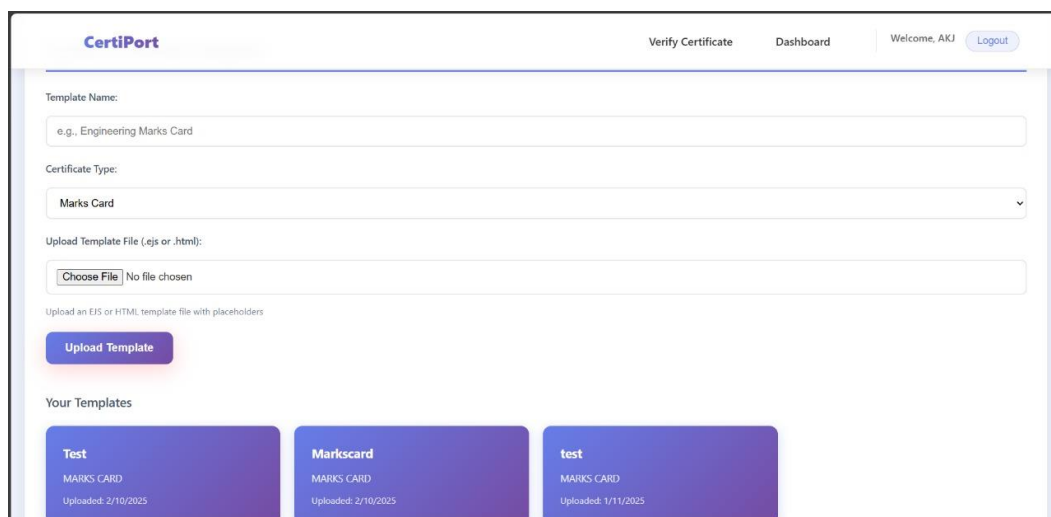
The University Dashboard serves as the control center for registered universities, allowing them to manage all certificate-related operations. It provides access to key modules such as uploading certificate templates, issuing certificates to students, and viewing previously issued certificates — ensuring a seamless and secure certificate management workflow within the blockchain-based system (refer fig. 5.6).



The screenshot shows the 'University Dashboard' for 'rohanbenny@kristujayanti.com'. The dashboard includes a 'Verified University' badge and a 'View Profile' button. Below the header, there are three main action buttons: 'Upload Templates', 'Issue Certificates', and 'Issued Certificates'. The 'Upload Certificate Template' section is active, showing a form with a 'Template Name' field (containing 'e.g., Engineering Marks Card') and a 'Certificate Type' dropdown menu (set to 'Marks Card').

Figure 5.6 Login Output Design

This interface allows universities to upload and manage certificate templates efficiently. The section below displays all previously uploaded templates with details like template name, type, and upload date, enabling easy selection and reuse for issuing new certificates (refer fig. 5.7).



The screenshot shows the 'Upload Certificate Template' form. It includes a 'Template Name' field (containing 'e.g., Engineering Marks Card'), a 'Certificate Type' dropdown menu (set to 'Marks Card'), and a file upload section with a 'Choose File' button and a 'No file chosen' message. Below the upload section, there is a section titled 'Your Templates' displaying a list of previously uploaded templates. The list contains three entries:

Template Name	Type	Upload Date
Test	MARKS CARD	Uploaded: 2/10/2025
Markscard	MARKS CARD	Uploaded: 2/10/2025
test	MARKS CARD	Uploaded: 1/11/2025

Figure 5.7 Upload Certificate Output Design

This interface displays the student's issued certificates in a visually organized layout. Each card shows essential details such as the certificate type, university name, course, issue date, and blockchain ID, along with options to view or download the certificate (refer fig. 5.8).

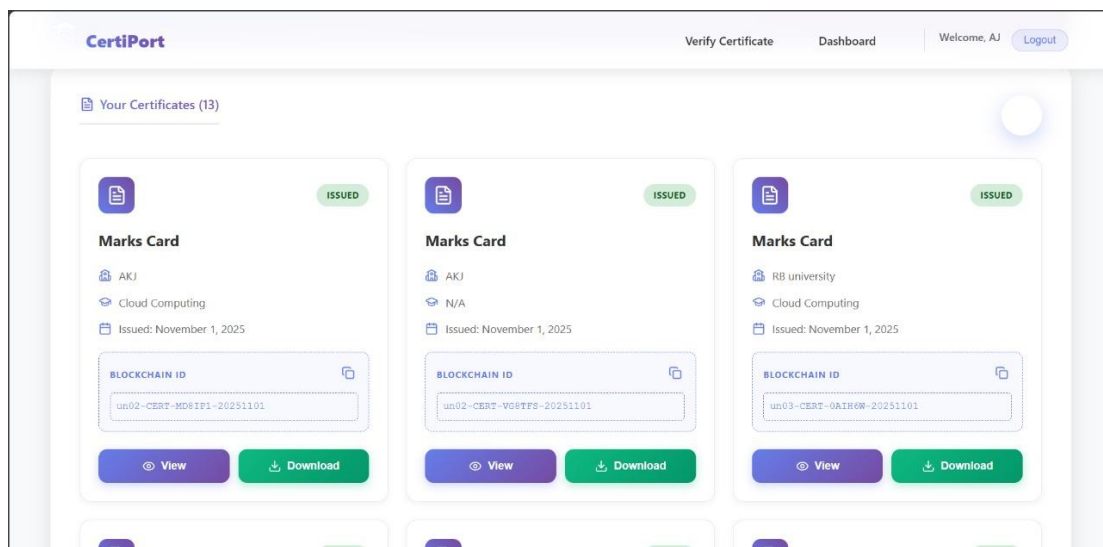


Figure 5.8 Issue Certificate Output Design

This interface shows the Admin Panel for managing pending university registration requests. It displays university details such as name, contact information, accreditation ID, address, and courses offered, along with options to approve or reject the request (refer fig 5.9).

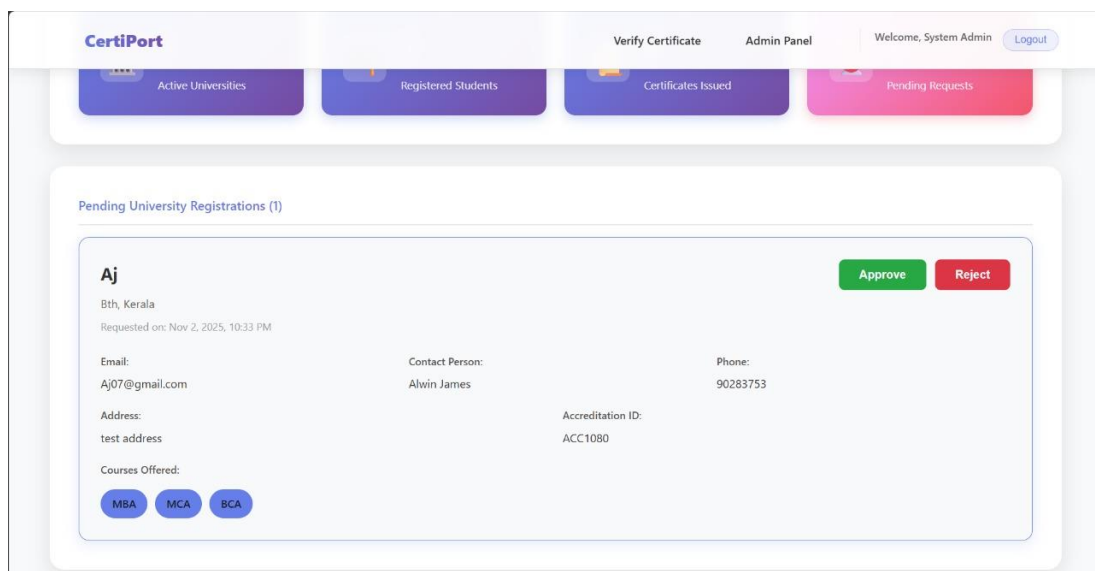


Figure 5.9 Create Account Output Design

5.4 Database Design

5.4.1 Users Table

Stores account information for all system users, including admins, universities, students, and verifiers (refer table 5.1).

Table 5.1 Users Table:

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each user.
email	VARCHAR(100)	User's registered email address.
password	VARCHAR(255)	Encrypted password using bcrypt.
role	ENUM('admin','university','student','verifier')	Defines user role.
universityDetails	JSON	Holds details for university users.
issuedCertificates	Array/JSON	Certificates issued by the university.
createdAt	DATETIME	Record creation timestamp.
updatedAt	DATETIME	Record last updated timestamp.
__v	INT	Schema version number.

5.4.2 Pending Requests Table

Manages university registration requests and their approval statuses (refer table 5.2).

Table 5.2 Pending Requests Table:

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for request.
email	VARCHAR(100)	University contact email.
password	VARCHAR(255)	Encrypted password.
role	ENUM('university')	Role associated with request.

universityDetails	JSON	Detailed information submitted by the university.
status	ENUM('pending','approved','rejected')	Current approval status.
reviewedBy	VARCHAR(50)	Admin reviewer name.
reviewDate	DATETIME	Date of review action.
createdAt	DATETIME	Timestamp when record was created.
updatedAt	DATETIME	Timestamp when record was updated.
universityId	VARCHAR(50)	Assigned University ID upon approval.

5.4.3 Templates Table

Contains reusable certificate templates with dynamic placeholders (refer table 5.3).

Table 5.3 Templates Table:

Field Name	Data Type	Description
_id	ObjectId	Unique template ID.
templateName	VARCHAR(100)	Template name for identification.
certificateType	VARCHAR(50)	Type of certificate (e.g., transfer, course).
universityId	VARCHAR(50)	ID of the university owning the template.
templatePath	VARCHAR(255)	File path to stored EJS template.
placeholders	JSON Array	List of fields to be dynamically filled.
isActive	BOOLEAN	Indicates if the template is active.
createdAt	DATETIME	Creation timestamp.
updatedAt	DATETIME	Last modified timestamp.

5.4.4 Certificates Table

Stores issued certificate records, along with blockchain verification hashes (refer table 5.4).

Table 5.4 Certificates Table:

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each certificate.
certificateId	VARCHAR(50)	Unique certificate code.
templateId	VARCHAR(50)	Reference to Templates table.
universityId	VARCHAR(50)	Reference to University in Users table.
studentName	VARCHAR(100)	Student's full name.
course	VARCHAR(100)	Course name or program title.
issueDate	DATE	Date when the certificate was issued.
blockchainHash	VARCHAR(255)	Hash of certificate stored on blockchain.
verificationStatus	BOOLEAN	Indicates blockchain verification success.
createdAt	DATETIME	Timestamp when created.
updatedAt	DATETIME	Timestamp when updated.

6. SOURCE CODE

landingPage.js

```
import React from 'react';
import { Link } from 'react-router-dom';
import '../styles/LandingPage.css';
const LandingPage = () => {
  return (
    <div className="landing-page">
      {/* Hero Section */}
      <section className="hero">
        <div className="hero-content">
          <div className="container">
            <div className="hero-text">
              <h1 className="hero-title">
                Blockchain-Powered <span className="highlight">Certificate Verification</span>
              </h1>
              <p className="hero-subtitle">
                Secure, tamper-proof, and instantly verifiable academic certificates.
                Eliminate fraud with blockchain technology and give students complete control over
                their credentials.
              </p>
              <div className="hero-buttons">
                <Link to="/register" className="btn btn-primary">Get Started</Link>
                <Link to="/login" className="btn btn-secondary">Login</Link>
              </div>
            </div>
          </div>
          <div className="hero-image">
            <div className="certificate-mockup">
              <div className="certificate">
                <div className="certificate-header">
```

```

    <div className="university-logo"></div>
    <h3>Digital Certificate</h3>
  </div>
  <div className="certificate-body">
    <div className="student-name">Xyz..</div>
    <div className="degree">Bachelor of Science</div>
    <div className="blockchain-id">ID: 0x1a2b3c4d...</div>
  </div>
  <div className="qr-code"></div>
</div>
</div>
</div>
</div>
</section>
{ /* Features Section */ }
<section className="features">
  <div className="container">
    <h2 className="section-title">Why Choose Our Platform?</h2>
    <div className="features-grid">
      <div className="feature-card">
        <div className="feature-icon"><img alt="lock icon" data-bbox="458 605 478 625"/></div>
        <h3>Fraud-Proof Security</h3>
        <p>Certificates stored on blockchain cannot be tampered with or faked. Complete authenticity guaranteed.</p>
      </div>
      <div className="feature-card">
        <div className="feature-icon"><img alt="lightning bolt icon" data-bbox="458 755 478 775"/></div>
        <h3>Instant Verification</h3>
        <p>Employers and institutions can verify certificates in seconds using blockchain ID or QR code.</p>
      </div>
      <div className="feature-card">

```

```

    <div className="feature-icon">🌐</div>
    <h3>Decentralized Storage</h3>
    <p>Certificates stored on IPFS ensure accessibility even if traditional servers go
down.</p>
  </div>
  <div className="feature-card">
    <div className="feature-icon">📱</div>
    <h3>Universal Access</h3>
    <p>Students can access their certificates anywhere, anytime, from any device with
internet.</p>
  </div>
</div>
{ /* How It Works Section */ }
<section className="how-it-works">
  <div className="container">
    <h2 className="section-title">How It Works</h2>
    <div className="steps">
      <div className="step">
        <div className="step-number">1</div>
        <div className="step-content">
          <h3>University Issues Certificate</h3>
          <p>Universities upload student data and generate blockchain-verified certificates
with unique IDs.</p>
        </div>
      </div>
      <div className="step">
        <div className="step-number">2</div>
        <div className="step-content">
          <h3>Blockchain Storage</h3>
          <p>Certificate data is recorded on blockchain and stored on IPFS for permanent,
tamper-proof storage.</p>

```

```

    </div>
  </div>
  <div className="step">
    <div className="step-number">3</div>
    <div className="step-content">
      <h3>Instant Verification</h3>
      <p>Anyone can verify certificate authenticity using the blockchain ID or scanning
the QR code.</p>
    </div>
  </div>
</div>
</div>
</section>
{ /* Target Audience Section */ }
<section className="target-audience">
  <div className="container">
    <h2 className="section-title">Built For Everyone</h2>
    <div className="audience-grid">
      <div className="audience-card">
        <div className="audience-icon"><img alt="University icon" data-bbox="475 570 498 588"/></div>
        <h3>Universities</h3>
        <ul>
          <li>Bulk certificate issuance</li>
          <li>Multiple certificate templates</li>
          <li>Excel upload support</li>
          <li>Automated PDF generation</li>
        </ul>
        <Link to="/register" className="audience-cta">Register University</Link>
      </div>
      <div className="audience-card">
        <div className="audience-icon"><img alt="Student icon" data-bbox="475 858 498 875"/></div>
        <h3>Students</h3>

```

```

<ul>
  <li>Access all certificates in one place</li>
  <li>Download blockchain-verified PDFs</li>
  <li>Share certificates securely</li>
  <li>No risk of document loss</li>
</ul>

<Link to="/login" className="audience-cta">Student Login</Link>
</div>
<div className="audience-card">
  <div className="audience-icon">👤</div>
  <h3>Employers</h3>
  <ul>
    <li>Instant certificate verification</li>
    <li>No need to contact universities</li>
    <li>100% authenticity guarantee</li>
    <li>Reduce hiring fraud</li>
  </ul>
  <Link to="/" className="audience-cta">Verify Certificate</Link>
</div>
</div>
</section>
{/* CTA Section */}
<section className="cta">
  <div className="container">
    <div className="cta-content">
      <h2>Ready to Secure Your Certificates?</h2>
      <p>Join thousands of universities and students already using blockchain-verified certificates.</p>
      <div className="cta-buttons">
        <Link to="/register" className="btn btn-primary btn-large">Get Started Now</Link>
        <Link to="/" className="btn btn-outline btn-large">Verify a Certificate</Link>
      </div>
    </div>
  </div>
</section>

```

```

    </div>
  </div>
</div>
</section>
{ /* Footer */ }
<footer className="footer">
  <div className="container">
    <div className="footer-content">
      <div className="footer-section">
        
        <p>Blockchain-powered certificate verification platform ensuring authenticity and
accessibility.</p>
      </div>
      <div className="footer-section">
        <h4>Quick Links</h4>
        <ul>
          <li><Link to="/login">Login</Link></li>
          <li><Link to="/register">Register</Link></li>
          <li><Link to="/">Verify Certificate</Link></li>
        </ul>
      </div>
      <div className="footer-section">
        <h4>For Universities</h4>
        <ul>
          <li><Link to="/register">Register Institution</Link></li>
          <li><a href="#bulk-upload">Bulk Certificate Upload</a></li>
          <li><a href="#templates">Certificate Templates</a></li>
        </ul>
      </div>
      <div className="footer-section">
        <h4>Support</h4>
        <ul>

```

```

    <li><a href="#help">Help Center</a></li>
    <li><a href="#contact">Contact Us</a></li>
    <li><a href="#api">API Documentation</a></li>
  </ul>
</div>
</div>
<div className="footer-bottom">
  <p>&copy; 2024 CertiPort. All rights reserved. | Powered by Blockchain
Technology</p>
</div>
</div>
</footer>
</div>
);
};
export default LandingPage;

```

adminDashboard.js

```

import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import '../styles/AdminDashboard.css';
const AdminDashboard = () => {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);
  const [pendingRequests, setPendingRequests] = useState([]);
  const [approvedUniversities, setApprovedUniversities] = useState([]);
  const [stats, setStats] = useState({
    totalUniversities: 0,
    totalStudents: 0,
    totalCertificates: 0,
    pendingRequests: 0
  });
};

```

```
const [actionLoading, setActionLoading] = useState({});
// Check if user is admin
useEffect(() => {
  const user = JSON.parse(localStorage.getItem('user') || 'null');
  const token = localStorage.getItem('token');
  if (!user || !token || user.role !== 'admin') {
    navigate('/login');
    return;
  }
  fetchDashboardData();
  // Set up real-time updates every 30 seconds
  const interval = setInterval(fetchDashboardData, 30000);
  return () => clearInterval(interval);
}, [navigate]);
const fetchDashboardData = async () => {
  try {
    setLoading(true);
    const token = localStorage.getItem('token');
    const [statsRes, pendingRes, universitiesRes] = await Promise.all([
      fetch('/api/admin/stats', {
        headers: { 'Authorization': `Bearer ${token}` }
      }),
      fetch('/api/admin/pending-requests', {
        headers: { 'Authorization': `Bearer ${token}` }
      }),
      fetch('/api/admin/universities', {
        headers: { 'Authorization': `Bearer ${token}` }
      })
    ]);
    if (statsRes.ok) {
      const statsData = await statsRes.json();
      setStats(statsData.stats);
    }
  }
}
```

```
    }
    if (pendingRes.ok) {
      const pendingData = await pendingRes.json();
      setPendingRequests(pendingData.requests);
    }
    if (universitiesRes.ok) {
      const universitiesData = await universitiesRes.json();
      setApprovedUniversities(universitiesData.universities);
    }
  } catch (error) {
    console.error('Error fetching dashboard data:', error);
  } finally {
    setLoading(false);
  }
};

const handleApproveUniversity = async (requestId) => {
  setActionLoading(prev => ({ ...prev, [requestId]: 'approving' }));
  try {
    const token = localStorage.getItem('token');
    const response = await fetch(`/api/admin/approve/${requestId}`, {
      method: 'PUT',
      headers: { 'Authorization': `Bearer ${token}` }
    });

    if (response.ok) {
      const data = await response.json();
      alert(`University approved successfully! University ID: ${data.universityId}`);
      fetchDashboardData(); // Refresh data
    } else {
      const errorData = await response.json();
      alert(errorData.msg || 'Error approving university');
    }
  }
```

```
    } catch (error) {
      console.error('Error approving university:', error);
      alert('Network error occurred');
    } finally {
      setActionLoading(prev => ({ ...prev, [requestId]: null }));
    }
  };

const handleRejectUniversity = async (requestId) => {
  const reason = prompt('Please provide a reason for rejection (optional):');
  setActionLoading(prev => ({ ...prev, [requestId]: 'rejecting' }));
  try {
    const token = localStorage.getItem('token');
    const response = await fetch(`/api/admin/reject/${requestId}`, {
      method: 'PUT',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ reason })
    });

    if (response.ok) {
      alert('University request rejected');
      fetchDashboardData(); // Refresh data
    } else {
      const errorData = await response.json();
      alert(errorData.msg || 'Error rejecting university');
    }
  } catch (error) {
    console.error('Error rejecting university:', error);
    alert('Network error occurred');
  } finally {
```

```
    setActionLoading(prev => ({ ...prev, [requestId]: null }));
  }
};

const handleRemoveUniversity = async (universityId, universityName) => {
  if (!window.confirm(`Are you sure you want to remove ${universityName}? This action
cannot be undone.`)) {
    return;
  }
  setActionLoading(prev => ({ ...prev, [universityId]: 'removing' }));
  try {
    const token = localStorage.getItem('token');
    const response = await fetch(`/api/admin/university/${universityId}`, {
      method: 'DELETE',
      headers: { 'Authorization': `Bearer ${token}` }
    });
    if (response.ok) {
      alert('University removed successfully');
      fetchDashboardData(); // Refresh data
    } else {
      const errorData = await response.json();
      alert(errorData.msg || 'Error removing university');
    }
  } catch (error) {
    console.error('Error removing university:', error);
    alert('Network error occurred');
  } finally {
    setActionLoading(prev => ({ ...prev, [universityId]: null }));
  }
};

return (
  <div className="admin-dashboard">
    <div className="admin-container">
```

```

    { /* Header */ }

    <div className="admin-header">
      <div className="header-content">
        
        <div>
          <h1 className="admin-title">Admin Dashboard</h1>
          <p className="admin-subtitle">Manage university registrations and system
overview</p>
        </div>
      </div>

      <button className="refresh-btn" onClick={fetchDashboardData}
disabled={loading}>
        {loading ? '⌂' : '⌂'}
      </button>
    </div>

    { /* Quick Stats */ }

    <div className="quick-stats">
      <h2 className="section-title">Quick Stats</h2>
      <div className="stats-grid">
        <div className="stat-card">
          <div className="stat-icon">🏛️</div>
          <div className="stat-content">
            <div className="stat-number">{stats.totalUniversities}</div>
            <div className="stat-label">Active Universities</div>
          </div>
        </div>

        <div className="stat-card pending">
          <div className="stat-icon">🔴⌂</div>
          <div className="stat-content">
            <div className="stat-number">{stats.pendingRequests}</div>
            <div className="stat-label">Pending Requests</div>
          </div>
        </div>
      </div>
    </div>

```

```

    </div>
  </div>
</div>
</div>
{ /* Pending University Registrations */}
<div className="admin-section">
  <h2 className="section-title">
    Pending University Registrations ({pendingRequests.length})
  </h2>
  {pendingRequests.length === 0 ? (
    <div className="empty-state">
      <div className="empty-icon">📋</div>
      <p>No pending university registrations.</p>
    </div>
  ) : (
    <div className="requests-list">
      {pendingRequests.map((request) => (
        <div key={request._id} className="request-card">
          <div className="request-header">
            <div className="university-info">
              <h3 className="university-name">
                {request.universityDetails.universityName}
              </h3>
              <p className="university-location">
                {request.universityDetails.city}, {request.universityDetails.state}
              </p>
              <p className="request-date">
                Requested on: {formatDate(request.createdAt)}
              </p>
            </div>
            <div className="request-actions">
              <button

```

```

        className="approve-btn"
        onClick={() => handleApproveUniversity(request._id)}
        disabled={actionLoading[request._id]}
    >
        {actionLoading[request._id] === 'approving' ? 'Approving...' : 'Approve'}
    </button>
</button>
    className="reject-btn"
    onClick={() => handleRejectUniversity(request._id)}
    disabled={actionLoading[request._id]}
    >
        {actionLoading[request._id] === 'rejecting' ? 'Rejecting...' : 'Reject'}
    </button>
</div>
</div>
<div className="detail-row">
    <div className="detail-item full-width">
        <strong>Courses Offered:</strong>
        <div className="courses-list">
            {request.universityDetails.coursesOffered.map((course, index) => (
                <span key={index} className="course-tag">{course}</span>
            ))}
        </div>
    </div>
</div>
{request.universityDetails.reasonForRegistration && (
    <div className="detail-row">
        <div className="detail-item full-width">
            <strong>Reason for Registration:</strong>
            <p className="reason-
text">{request.universityDetails.reasonForRegistration}</p>
        </div>
    </div>

```

```

        </div>
    })
</div>
</div>
    )})
</div>
})
</div>
{ /* All Universities */
<div className="admin-section">
    <h2 className="section-title">
        All Universities ( {approvedUniversities.length} )
    </h2>
    {approvedUniversities.length === 0 ? (
        <div className="empty-state">
            <div className="empty-icon">🏛️</div>
            <p>No universities found.</p>
        </div>
    ) : (
        <div className="universities-grid">
            {approvedUniversities.map((university) => (
                <div key={university._id} className="university-card">
                    <div className="university-header">
                        <div className="university-main-info">
                            <h3 className="university-name">
                                {university.universityDetails.universityName}
                            </h3>
                            <p className="university-id">
                                ID: {university.universityDetails.universityId}
                            </p>
                            <p className="university-location">
                                {university.universityDetails.city}, {university.universityDetails.state}

```

```

    </p>
  </div>

  <button
    className="remove-btn"
    onClick={() => handleRemoveUniversity(university._id,
university.universityDetails.universityName)}
    disabled={actionLoading[university._id]}
    title="Remove University"
  >
    {actionLoading[university._id] === 'removing' ? '...' : '×'}
  </button>
</div>

<div className="university-contact">
  <p><strong>Contact:</strong>
{university.universityDetails.contactPersonName}</p>
  <p><strong>Email:</strong> {university.email}</p>
  <p><strong>Phone:</strong> {university.universityDetails.contactNumber}</p>
</div>

<div className="courses-preview">
  <strong>Courses:</strong>
  <div className="courses-list">
    {university.universityDetails.coursesOffered.slice(0, 3).map((course, index) =>
(
      <span key={index} className="course-tag small">{course}</span>
    ))}
    {university.universityDetails.coursesOffered.length > 3 && (
      <span className="course-tag small more">
        + {university.universityDetails.coursesOffered.length - 3} more
      </span>
    )}
  </div>
</div>

```

```
        </div>

    )})
</div>

})
</div>

</div>

</div>

);

};

export default AdminDashboard;
```

register.js

```
import React, { useState, useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import '../styles/Register.css';
```

```
const Register = () => {
  const navigate = useNavigate();

  // Form state
  const [formData, setFormData] = useState({
    email: "",
    password: "",
    confirmPassword: "",
    role: "",
    // Student fields
    fullName: "",
    dateOfBirth: "",
    program: "",
    enrollmentNumber: "",
    universityName: "",
    yearOfAdmission: "",
    yearOfGraduation: "",
    // PG specific fields
    pgEnrollmentNumber: "",
    pgUniversityName: "",
    pgYearOfAdmission: "",
    pgYearOfGraduation: "",
    // University fields
    universityNameField: "",
    contactPersonName: "",
    contactNumber: "",
    officialAddress: "",
    city: "",
```

```
state: ",
country: ",
accreditationId: ",
coursesOffered: ",
reasonForRegistration: "
});

const [errors, setErrors] = useState({});
const [loading, setLoading] = useState(false);
const [message, setMessage] = useState("");
const [showPassword, setShowPassword] = useState(false);
const [showConfirmPassword, setShowConfirmPassword] = useState(false);
const [approvedUniversities, setApprovedUniversities] = useState([]);

// Fetch approved universities for student dropdown
useEffect(() => {
  if (formData.role === 'student') {
    fetchApprovedUniversities();
  }
}, [formData.role]);

const fetchApprovedUniversities = async () => {
  try {
    const response = await fetch('/api/public/approved-universities');
    if (response.ok) {
      const data = await response.json();
      setApprovedUniversities(data.universities || []);
    }
  } catch (error) {
    console.error('Error fetching universities:', error);
  }
};

const validateForm = () => {
  const newErrors = {};

  // Common validations
  if (!formData.email) newErrors.email = 'Email is required';
  else if (!/^S+@\S+\.\S+/.test(formData.email)) newErrors.email = 'Invalid email format';

  if (!formData.password) newErrors.password = 'Password is required';
  else if (formData.password.length < 8) newErrors.password = 'Password must be at least 8 characters';

  if (formData.password !== formData.confirmPassword) {
    newErrors.confirmPassword = 'Passwords do not match';
  }

  if (!formData.role) newErrors.role = 'Please select a role';

  // Student-specific validations
```

```
if (formData.role === 'student') {
  if (!formData.fullName) newErrors.fullName = 'Full name is required';
  if (!formData.dateOfBirth) newErrors.dateOfBirth = 'Date of birth is required';
  if (!formData.program) newErrors.program = 'Program selection is required';
  if (!formData.enrollmentNumber) newErrors.enrollmentNumber = 'Enrollment number is required';
  if (!formData.universityName) newErrors.universityName = 'University selection is required';
  if (!formData.yearOfAdmission) newErrors.yearOfAdmission = 'Year of admission is required';
  if (!formData.yearOfGraduation) newErrors.yearOfGraduation = 'Year of graduation is required';

  // PG specific validations
  if (formData.program === 'pg') {
    if (!formData.pgEnrollmentNumber) newErrors.pgEnrollmentNumber = 'PG enrollment number is required';
    if (!formData.pgUniversityName) newErrors.pgUniversityName = 'PG university selection is required';
    if (!formData.pgYearOfAdmission) newErrors.pgYearOfAdmission = 'PG year of admission is required';
    if (!formData.pgYearOfGraduation) newErrors.pgYearOfGraduation = 'PG year of graduation is required';
  }
}

// University-specific validations
if (formData.role === 'university') {
  if (!formData.universityNameField) newErrors.universityNameField = 'University name is required';
  if (!formData.contactPersonName) newErrors.contactPersonName = 'Contact person name is required';
  if (!formData.contactNumber) newErrors.contactNumber = 'Contact number is required';
  if (!formData.officialAddress) newErrors.officialAddress = 'Official address is required';
  if (!formData.city) newErrors.city = 'City is required';
  if (!formData.state) newErrors.state = 'State is required';
  if (!formData.country) newErrors.country = 'Country is required';
  if (!formData.accreditationId) newErrors.accreditationId = 'Accreditation ID is required';
  if (!formData.coursesOffered) newErrors.coursesOffered = 'Courses offered is required';
}

setErrors(newErrors);
return Object.keys(newErrors).length === 0;
};

const handleSubmit = async (e) => {
  e.preventDefault();

  if (!validateForm()) {
    return;
  }
}
```

```

    }

    setLoading(true);
    setMessage("");

    try {
      const response = await fetch('/api/auth/register', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(formData),
      });

      const data = await response.json();

      if (response.ok) {
        if (formData.role === 'university') {
          setMessage('University registration submitted successfully! Please wait for admin approval.');
```

approval.');

```

        } else {
          setMessage('Registration successful! You can now login.');
```

setMessage('Registration successful! You can now login.');

```

          setTimeout(() => {
            navigate('/login');
          }, 2000);
        }
      } else {
        setMessage(data.msg || 'Registration failed');
      }
    } catch (error) {
      setMessage('Network error. Please try again.');
```

setMessage('Network error. Please try again.');

```

      console.error('Registration error:', error);
    } finally {
      setLoading(false);
    }
  };

  const currentYear = new Date().getFullYear();
  const years = Array.from({ length: 50 }, (_, i) => currentYear - i);

  return (
    <div className="register-page">
      <div className="register-container">
        <div className="register-header">
          <div className="register-logo">
            
          </div>
          <h1 className="register-title">Create Account</h1>
          <p className="register-subtitle">Join the blockchain-powered certificate verification platform</p>

```

```

</div>

<form className="register-form" onSubmit={handleSubmit}>
  {message && (
    <div className={`message ${formData.role === 'university' &&
message.includes('submitted') ? 'success-message' : message.includes('successful') ? 'success-
message' : 'error-message'} `}>
      {message}
    </div>
  )}

  {/ * Basic Info */}
  <div className="form-section">
    <h3 className="section-title">Account Information</h3>

    <div className="form-group">
      <label className="form-label">Email Address</label>
      <div className="form-input-wrapper">
        <input
          type="email"
          name="email"
          className={`form-input ${errors.email ? 'error' : ''}`}
          placeholder="Enter your email address"
          value={formData.email}
          onChange={handleInputChange}
        />
        <span className="input-icon">✉️</span>
      </div>
      {errors.email && <span className="error-text">{errors.email}</span>}
    </div>

    <div className="form-row">
      <div className="form-group">
        <label className="form-label">Password</label>
        <div className="form-input-wrapper">
          <input
            type={showPassword ? "text" : "password"}
            name="password"
            className={`form-input ${errors.password ? 'error' : ''}`}
            placeholder="Enter password"
            value={formData.password}
            onChange={handleInputChange}
          />
          <span className="input-icon">🔒</span>
        </div>
        <button
          type="button"
          className="password-toggle"
          onClick={() => setShowPassword(!showPassword)}
        >
          {showPassword ? "👁️" : "👁️"}
        </button>
      </div>
    </div>
  </div>

```

```

    </button>
  </div>
  {errors.password && <span className="error-text">{errors.password}</span>}
</div>

<div className="form-group">
  <label className="form-label">Confirm Password</label>
  <div className="form-input-wrapper">
    <input
      type={showConfirmPassword ? "text" : "password"}
      name="confirmPassword"
      className={`form-input ${errors.confirmPassword ? 'error' : ''}`}
      placeholder="Confirm password"
      value={formData.confirmPassword}
      onChange={handleInputChange}
    />
    <span className="input-icon">🔒</span>
    <button
      type="button"
      className="password-toggle"
      onClick={() => setShowConfirmPassword(!showConfirmPassword)}
    >
      {showConfirmPassword ? "👁" : "👁"}
    </button>
  </div>
  {errors.confirmPassword && <span className="error-
text">{errors.confirmPassword}</span>}
</div>
</div>
</div>

{/* Student Fields */}
{formData.role === 'student' && (
  <div className="form-section">
    <h3 className="section-title">Student Information</h3>

    <div className="form-group">
      <label className="form-label">Full Name (As per university records)</label>
      <input
        type="text"
        name="fullName"
        className={`form-input ${errors.fullName ? 'error' : ''}`}
        placeholder="Enter your full name"
        value={formData.fullName}
        onChange={handleInputChange}
      />
      {errors.fullName && <span className="error-text">{errors.fullName}</span>}
    </div>

    <div className="form-row">

```

```

<div className="form-group">
  <label className="form-label">Date of Birth</label>
  <input
    type="date"
    name="dateOfBirth"
    className={`form-input ${errors.dateOfBirth ? 'error' : ''}`}
    value={formData.dateOfBirth}
    onChange={handleInputChange}
  />
  {errors.dateOfBirth && <span className="error-
text">{errors.dateOfBirth}</span>}
</div>

<div className="form-group">
  <label className="form-label">Program</label>
  <select
    name="program"
    className={`form-input ${errors.program ? 'error' : ''}`}
    value={formData.program}
    onChange={handleInputChange}
  >
    <option value="">Select program</option>
    <option value="ug">Undergraduate (UG)</option>
    <option value="pg">Postgraduate (PG)</option>
  </select>
  {errors.program && <span className="error-text">{errors.program}</span>}
</div>
</div>

{/* UG Details */}
{(formData.program === 'ug' || formData.program === 'pg') && (
  <div className="education-section">
    <h4 className="subsection-title">
      {formData.program === 'pg' ? 'Undergraduate Details' : 'Academic Details'}
    </h4>
    <div className="form-group">
      <label className="form-label">University Name</label>
      <select
        name="universityName"
        className={`form-input ${errors.universityName ? 'error' : ''}`}
        value={formData.universityName}
        onChange={handleInputChange}
      >
        <option value="">Select university</option>
        {approvedUniversities.map((uni, index) => (
          <option key={index} value={uni.name}>
            {uni.name}
          </option>
        ))}
      </select>
    </div>
  </div>
)}

```

```

        {errors.universityName && <span className="error-
text">{errors.universityName}</span>}
      </div>
    </div>

    <div className="form-row">
      <div className="form-group">
        <label className="form-label">Year of Admission</label>
        <select
          name="yearOfAdmission"
          className={`form-input ${errors.yearOfAdmission ? 'error' : ''}`}
          value={formData.yearOfAdmission}
          onChange={handleInputChange}
        >
          <option value="">Select year</option>
          {years.map(year => (
            <option key={year} value={year}>{year}</option>
          ))}
        </select>
        {errors.yearOfAdmission && <span className="error-
text">{errors.yearOfAdmission}</span>}
      </div>
    </div>
  </div>
)}
{/* PG Details */}
{formData.program === 'pg' && (
  <div className="education-section">
    <h4 className="subsection-title">Postgraduate Details</h4>

    <div className="form-row">
      <div className="form-group">
        <label className="form-label">PG Enrollment/Roll Number</label>
        <input
          type="text"
          name="pgEnrollmentNumber"
          className={`form-input ${errors.pgEnrollmentNumber ? 'error' : ''}`}
          placeholder="Enter PG enrollment number"
          value={formData.pgEnrollmentNumber}
          onChange={handleInputChange}
        />
        {errors.pgEnrollmentNumber && <span className="error-
text">{errors.pgEnrollmentNumber}</span>}
      </div>

      <div className="form-group">
        <label className="form-label">PG University Name</label>
        <select
          name="pgUniversityName"
          className={`form-input ${errors.pgUniversityName ? 'error' : ''}`}

```

```

        value={formData.pgUniversityName}
        onChange={handleInputChange}
      >
      <option value="">Select PG university</option>
      {approvedUniversities.map((uni, index) => (
        <option key={index} value={uni.name}>
          {uni.name}
        </option>
      ))}
    </select>
    {errors.pgUniversityName && <span className="error-
text">{errors.pgUniversityName}</span>}
  </div>
</div>

<div className="form-row">
  <div className="form-group">
    <label className="form-label">PG Year of Admission</label>
    <select
      name="pgYearOfAdmission"
      className={`form-input ${errors.pgYearOfAdmission ? 'error' : ''}`}
      value={formData.pgYearOfAdmission}
      onChange={handleInputChange}
    >
      <option value="">Select year</option>
      {years.map(year => (
        <option key={year} value={year}>{year}</option>
      ))}
    </select>
    {errors.pgYearOfAdmission && <span className="error-
text">{errors.pgYearOfAdmission}</span>}
  </div>

  <div className="form-group">
    <label className="form-label">PG Year of Graduation</label>
    <select
      name="pgYearOfGraduation"
      className={`form-input ${errors.pgYearOfGraduation ? 'error' : ''}`}
      value={formData.pgYearOfGraduation}
      onChange={handleInputChange}
    >
      <option value="">Select year</option>
      {years.map(year => (
        <option key={year} value={year}>{year}</option>
      ))}
    </select>
    {errors.pgYearOfGraduation && <span className="error-
text">{errors.pgYearOfGraduation}</span>}
  </div>
</div>

```

```

    </div>
  )}
</div>
)}

{/* University Fields */}
{formData.role === 'university' && (
  <div className="form-section">
    <h3 className="section-title">University Information</h3>

    <div className="form-group">
      <label className="form-label">University Name</label>
      <input
        type="text"
        name="universityNameField"
        className={`form-input ${errors.universityNameField ? 'error' : ''}`}
        placeholder="Enter university name"
        value={formData.universityNameField}
        onChange={handleInputChange}
      />
      {errors.universityNameField && <span className="error-
text">{errors.universityNameField}</span>}
    </div>

    <div className="form-row">
      <div className="form-group">
        <label className="form-label">Contact Person Name</label>
        <input
          type="text"
          name="contactPersonName"
          className={`form-input ${errors.contactPersonName ? 'error' : ''}`}
          placeholder="Registrar/Admin name"
          value={formData.contactPersonName}
          onChange={handleInputChange}
        />
        {errors.contactPersonName && <span className="error-
text">{errors.contactPersonName}</span>}
      </div>

      <div className="form-group">
        <label className="form-label">Contact Number</label>
        <input
          type="tel"
          name="contactNumber"
          className={`form-input ${errors.contactNumber ? 'error' : ''}`}
          placeholder="Enter contact number"
          value={formData.contactNumber}
          onChange={handleInputChange}
        />

```

```

        {errors.contactNumber} && <span className="error-
text">{errors.contactNumber}</span>
      </div>
    </div>

    <div className="form-group">
      <label className="form-label">Official University Address</label>
      <input
        type="text"
        name="officialAddress"
        className={`form-input ${errors.officialAddress ? 'error' : ''}`}
        placeholder="Enter complete address"
        value={formData.officialAddress}
        onChange={handleInputChange}
      />
      {errors.officialAddress} && <span className="error-
text">{errors.officialAddress}</span>
    </div>

    <div className="form-row">
      <div className="form-group">
        <label className="form-label">City</label>
        <input
          type="text"
          name="city"
          className={`form-input ${errors.city ? 'error' : ''}`}
          placeholder="Enter city"
          value={formData.city}
          onChange={handleInputChange}
        />
        {errors.city} && <span className="error-text">{errors.city}</span>
      </div>

      <div className="form-group">
        <label className="form-label">State</label>
        <input
          type="text"
          name="state"
          className={`form-input ${errors.state ? 'error' : ''}`}
          placeholder="Enter state"
          value={formData.state}
          onChange={handleInputChange}
        />
        {errors.state} && <span className="error-text">{errors.state}</span>
      </div>

      <div className="form-group">
        <label className="form-label">Country</label>
        <input
          type="text"

```

```

        name="country"
        className={`form-input ${errors.country ? 'error' : ''}`}
        placeholder="Enter country"
        value={formData.country}
        onChange={handleInputChange}
      />
      {errors.country && <span className="error-text">{errors.country}</span>}
    </div>
  </div>

  <div className="form-group">
    <label className="form-label">Accreditation/Registration ID</label>
    <input
      type="text"
      name="accreditationId"
      className={`form-input ${errors.accreditationId ? 'error' : ''}`}
      placeholder="Enter accreditation ID"
      value={formData.accreditationId}
      onChange={handleInputChange}
    />
    {errors.accreditationId && <span className="error-
text">{errors.accreditationId}</span>}
  </div>

  <div className="form-group">
    <label className="form-label">Courses Offered</label>
    <textarea
      name="coursesOffered"
      className={`form-input ${errors.coursesOffered ? 'error' : ''}`}
      placeholder="List courses separated by commas (e.g., Computer Science,
Mechanical Engineering, MBA)"
      rows="3"
      value={formData.coursesOffered}
      onChange={handleInputChange}
    />
    {errors.coursesOffered && <span className="error-
text">{errors.coursesOffered}</span>}
  </div>

  <div className="form-group">
    <label className="form-label">Reason for Registration (Optional)</label>
    <textarea
      name="reasonForRegistration"
      className="form-input"
      placeholder="Brief reason for joining the platform"
      rows="3"
      value={formData.reasonForRegistration}
      onChange={handleInputChange}
    />
  </div>

```

```
    </div>
  )}

  <button
    type="submit"
    className="register-button"
    disabled={loading}
  >
    {loading ? 'Creating Account...' : 'Create Account'}
  </button>

  <div className="login-link">
    Already have an account? <Link to="/login">Sign in here</Link>
  </div>

  <div className="security-badge">
    Your information is encrypted and secure
  </div>
</form>
</div>
</div>
);
};

export default Register;
```

7. SYSTEM TESTING

Admin Module Test Cases

This section focuses on testing the Admin Module of the Blockchain-Based University Certificate Issuance and Verification System. The primary objective of these test cases is to ensure that the admin can successfully manage universities, authenticate credentials, and maintain proper control over system operations. The validations performed in these test cases include verifying login credentials, error message handling, and university approval workflows (refer table 7.1).

Table 7.1 Admin Module Test Cases:

Test ID	Test Condition	Test Description	Test Data	Expected Result	Actual Result	Final Result
TC01	Admin Login	Verify that the admin can log in with valid credentials.	Email: admin@system.com, Password: Admin@123	Admin successfully logs in and dashboard is displayed.	As Expected	Pass
TC02	Invalid Login Attempt	Verify system behavior for invalid admin credentials.	Email: admin@system.com, Password: wrong123	System displays error message 'Invalid credentials'.	As Expected	Pass
TC03	Approve University	Verify that admin can approve pending university registration.	University ID: un03, Name: RB University	University status changes to 'Approved' and confirmation email is sent.	As Expected	Pass

University Module Test Cases

This section focuses on testing the University Module of the Blockchain-Based Certificate Issuance and Verification System (refer fig 10.4). The university users can successfully log in, upload templates, issue certificates, manage templates effectively. Validation ensures that data entered by the university—such as templates, certificate details, and issued records—is properly verified, stored, and synchronized with the blockchain and IPFS systems (refer table 7.2).

Table 7.2 University Module Test Cases:

Test ID	Test Condition	Test Description	Test Data	Expected Result	Actual Result	Final Result
TC04	University Login	Verify that approved university login is successful.	Email: rohanbenny@kristujayanti.com, Password: RB@2025	University dashboard loads successfully.	As Expected	Pass
TC05	Upload Certificate Template	Verify that university can upload an EJS template for certificates.	Template Name: Degree Certificate, File: degree.ejs	Template uploaded successfully and visible in the template list.	As Expected	Pass
TC06	Issue Certificate	Verify that the university can issue a certificate using uploaded template.	Student Name: Mia Turner, Email: miaturner@strivo.edu, Template: degree.ejs	Certificate issued, uploaded to IPFS, and metadata stored on blockchain.	As Expected	Pass
TC07	Deactivate Template	Verify that the university can deactivate a template.	Template: degree.ejs	Template status changes to inactive and cannot be used for new issuance.	As Expected	Pass

Student Module Test Cases

This section validates the functionality of the Student Module in the Blockchain-Based Certificate Issuance and Verification System. The main objective of this testing phase is to ensure that students can register, access, and manage their issued certificates securely (refer fig 10.5).

It also verifies that pre-issued certificates are automatically linked upon registration and that blockchain data integrity is maintained during certificate retrieval (refer table 7.3).

Table 7.3 Student Module Test Cases:

Test ID	Test Condition	Test Description	Test Data	Expected Result	Actual Result	Final Result
TC08	Student Registration	Verify that a student can register using a valid email.	Email: miaturner@strivo.edu, Password: Mia@2025	Student account created successfully.	As Expected	Pass
TC09	Auto-Link Issued Certificates	Verify that pre-issued certificates appear after registration.	Email: miaturner@strivo.edu	All certificates linked to this email appear in the student dashboard.	As Expected	Pass
TC10	View and Download Certificate	Verify that student can view and download their issued certificates.	Student: Mia Turner, Certificate ID: un03-CERT-MC456RB-20251101	Certificate details and download link displayed successfully.	As Expected	Pass

Verifier Module Test Cases

This section focuses on testing the Verifier Module, which enables third-party users, such as employers or institutions, to verify the authenticity of academic certificates using blockchain technology (refer fig 10.6). The objective is to ensure that the verification process accurately retrieves certificate metadata, displays verification results, and properly handles invalid or delayed blockchain responses. Validation is primarily based on blockchain ID integrity, IPFS data retrieval, and real-time synchronization between the blockchain and the verification interface (refer table 7.4).

Table 7.4 Verifier Module Test Cases:

Test ID	Test Condition	Test Description	Test Data	Expected Result	Actual Result	Final Result
TC11	Verify Valid Certificate	Verify that verifier can validate certificate using blockchain ID.	Blockchain ID: un03-CERT-MC456RB-20251101	Certificate metadata fetched and PDF preview displayed.	As Expected	Pass
TC12	Verify Invalid Certificate	Verify that verifier gets an error message for invalid blockchain ID.	Blockchain ID: INVALID-999	System displays 'Invalid or Fake Certificate' message.	As Expected	Pass
TC13	Blockchain Response Handling	Verify that verification works even if IPFS retrieval is delayed.	Blockchain ID: un03-CERT-TC789RB-20251101	System displays certificate metadata once IPFS file loads.	As Expected	Pass

8. SYSTEM IMPLEMENTATION

8.1 System Implementation

The Blockchain-Based University Certificate Issuance and Verification System was developed using a modular approach that ensures efficient integration, scalability, and security. The primary objective of implementation was to integrate blockchain, IPFS, and web-based components into a seamless and secure certificate management platform.

Frontend Implementation

The frontend was implemented using React.js for creating an interactive and responsive web interface. It allows users (Admin, University, Student, and Verifier) to perform different operations through role-based dashboards.

- **Admin Interface:** Enables admin users to approve universities and monitor system usage.
- **University Interface:** Provides universities with tools to upload certificate templates, fill student details, and issue certificates.
- **Student Interface:** Allows students to view, download, and verify their certificates stored on the blockchain.
- **Verifier Interface:** Permits third-party organizations to verify the authenticity of any certificate using the blockchain ID.

APIs are integrated using Axios for smooth communication with the backend, while dynamic routing ensures user-specific navigation.

Backend Implementation

The backend was developed using Node.js and Express.js, which serve as the backbone of the system. It handles API routing, certificate generation, database interactions, and communication with the blockchain network. The backend performs the following tasks:

- Generates a certificate PDF from uploaded templates.
- Uploads the generated certificate to IPFS through Pinata SDK.
- Stores metadata (IPFS hash, certificate ID, and issue date) on the blockchain via Ether.js.

- Saves user and certificate details in MongoDB for system-level access and tracking.

Error handling, authentication, and secure data management were implemented using middleware and encryption techniques to ensure safe user interactions.

Blockchain Integration

The blockchain layer was implemented using Solidity Smart Contracts deployed through Hardhat. Each issued certificate is recorded immutably on the blockchain with attributes such as certificate ID, IPFS hash, university ID, and issue timestamp. This ensures that no certificate can be altered or deleted once published. Smart contracts expose key functions such as:

- `issueCertificate()` for certificate creation,
- `getCertificateDetails()` for retrieval, and
- `verifyCertificate()` for public verification.

Communication with the blockchain is managed using `Ether.js`, enabling secure and efficient execution of contract functions from the backend.

IPFS and Database Integration

The InterPlanetary File System (IPFS), integrated via the Pinata API, is responsible for decentralized certificate storage. Each certificate file is uploaded, and the returned IPFS hash is linked to the blockchain record.

Meanwhile, MongoDB is used to store structured data, including user profiles, template metadata, and certificate mappings. It provides efficient retrieval and ensures synchronization between system modules.

8.2 System Deployment

The deployment process ensures that the system runs seamlessly in a production-like environment with secure access and reliable performance.

Deployment Environment

- **Frontend:** Deployed on a cloud hosting platform such as Vercel or Netlify, allowing continuous integration and deployment for React-based applications.
- **Backend:** Hosted on AWS EC2 or Render, ensuring scalability and high availability.

- Database: Managed through MongoDB Atlas, offering a secure, cloud-based database service.
- Blockchain: Smart contracts deployed on the Ethereum (or Polygon Mumbai Testnet) network using Hardhat for testing and contract management.
- IPFS: Certificates stored and pinned using Pinata Cloud for persistent decentralized storage.

Deployment Procedure

- Smart Contract Deployment:
Using Hardhat, smart contracts were compiled and deployed to the test network. Contract addresses and ABI files were saved for backend communication.
- Backend Deployment:
The Node.js server was hosted on a cloud service with environment variables configured for database connections, blockchain credentials, and Pinata API keys.
- Frontend Deployment:
The React frontend was built using the npm run build command and deployed to a hosting service. API base URLs were configured to connect with the deployed backend.
- Database Configuration:
MongoDB Atlas was configured with secured access credentials and linked to the backend via environment variables.

The complete deployment ensures a fully functional, decentralized certificate management system accessible through a secure web interface. The integration of blockchain, IPFS, and web technologies guarantees transparency, immutability, and scalability. This system enables universities to issue verifiable certificates, students to access them securely, and organizations to verify authenticity effortlessly — forming a trust-based digital ecosystem.

9. CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The Blockchain-Based University Certificate Issuance and Verification System marks a significant step toward digital transformation in the education sector by providing a reliable, transparent, and decentralized framework for managing academic credentials. The system effectively addresses the long-standing challenges of certificate forgery, data loss, and time-consuming verification processes that plague traditional, paper-based systems. By leveraging the combined power of Blockchain and IPFS (InterPlanetary File System), the solution ensures that all issued certificates are immutable, permanently accessible, and easily verifiable by anyone with the blockchain ID.

One of the key achievements of this project lies in its automation and decentralization. The platform eliminates the need for intermediaries and manual validation steps by recording certificate data directly on the blockchain. This not only enhances transparency but also significantly reduces administrative overhead for universities. The integration of customizable certificate templates allows institutions to maintain branding and flexibility while standardizing the issuance process. Furthermore, the seamless linking of certificates with students' email IDs ensures that academic records remain accessible for life, even if students change institutions or lose their physical copies.

The frontend and backend design demonstrate a thoughtful combination of modern web technologies. The React.js frontend ensures a responsive and user-friendly interface, while Node.js and Express.js provide a robust and scalable backend infrastructure. The Solidity smart contracts ensure security and immutability of certificate records on the Ethereum blockchain, while MongoDB efficiently handles user data and template storage. The use of Ether.js and Hardhat streamlines blockchain interactions and smart contract deployment, making the system both developer-friendly and reliable in operation.

This project also emphasizes user accessibility and trust. Students can instantly access and download their verified certificates, while third-party verifiers — such as employers or other educational institutions — can authenticate documents within seconds without depending on manual confirmation from universities. The system embodies the principles of transparency, decentralization, and efficiency, making it a strong candidate for real-world implementation across educational ecosystems.

In essence, the project not only demonstrates the practicality of blockchain technology in academic record management but also sets the foundation for a future-ready digital credential ecosystem. By adopting this system, universities can significantly reduce administrative workloads, increase the credibility of issued certificates, and contribute to a globally recognized framework for academic verification. This initiative paves the way for a more secure, paperless, and interoperable educational environment, empowering both institutions and learners in the digital age.

9.2 Future Enhancement

- **Mobile Application Integration**

A dedicated mobile app can be developed for students and verifiers, allowing on-the-go access and verification of certificates using blockchain IDs.

- **Integration with National or Global Education Networks:**

The system can be expanded to integrate with national or international educational databases to enable cross-university verification and global interoperability of academic records.

- **Cross-Institutional Collaboration**

Expanding the system to support multiple universities and cross-border verification can create a global network for trusted academic credential validation.

In the future, this system can evolve into a comprehensive credential management framework connecting universities, employers, and accreditation bodies on a unified blockchain network. With continued advancements in decentralized technologies, such solutions will redefine the way educational qualifications are issued, verified, and preserved. This project demonstrates a strong foundation upon which more intelligent, interoperable, and globally accepted digital certificate systems can be built. Ultimately, it stands as a proof of how blockchain can bring trust, transparency, and permanence to academic verification processes worldwide.

10. APPENDICES

10.1 Screenshots

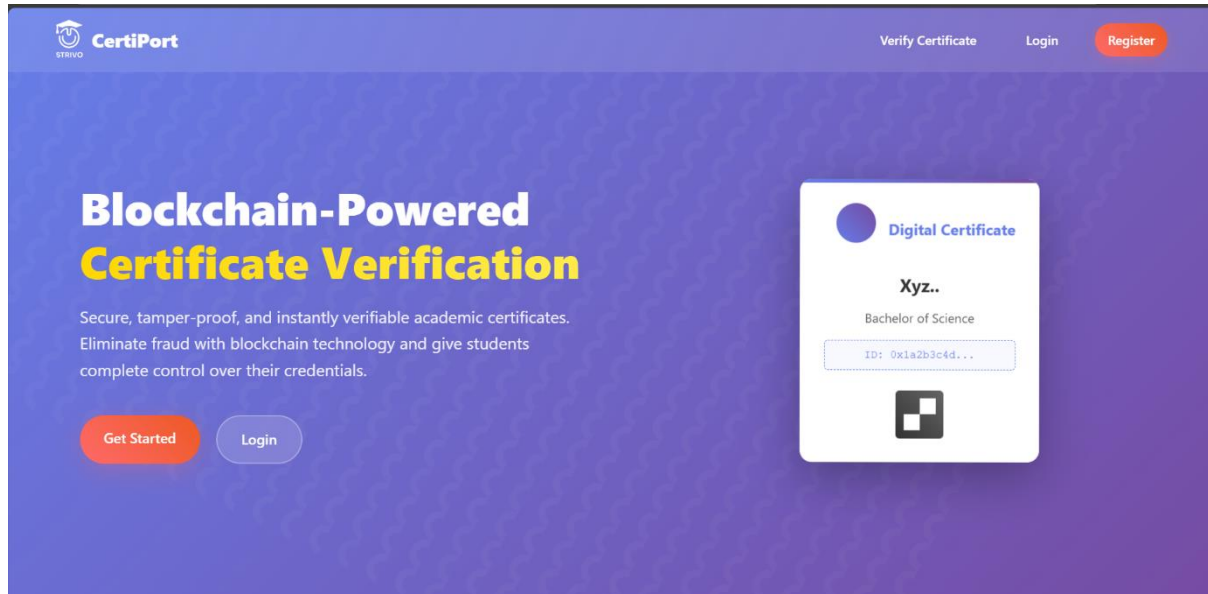


Figure 10.1 Landing Page

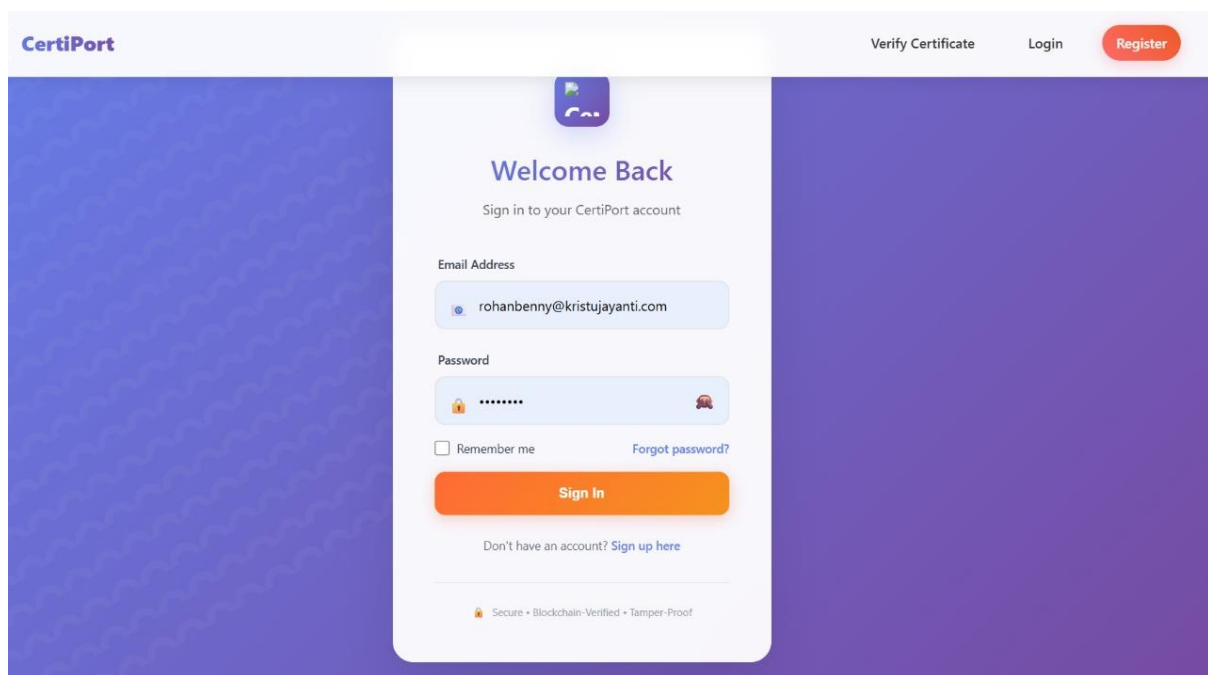


Figure 10.2 Login Page

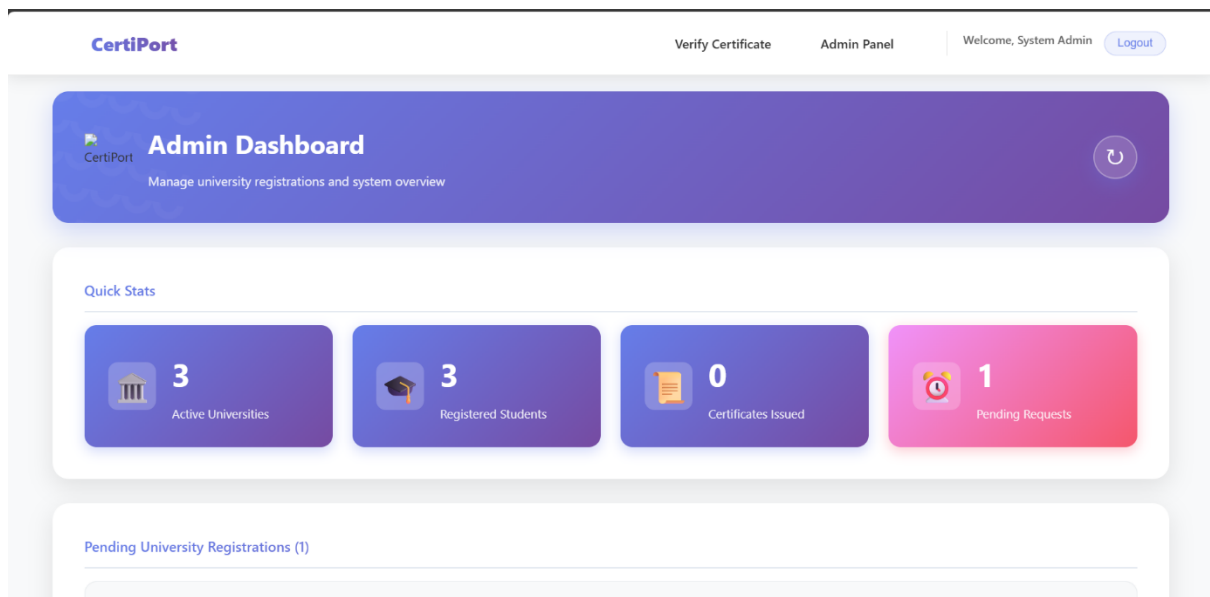


Figure 10.3 Admin Dashboard

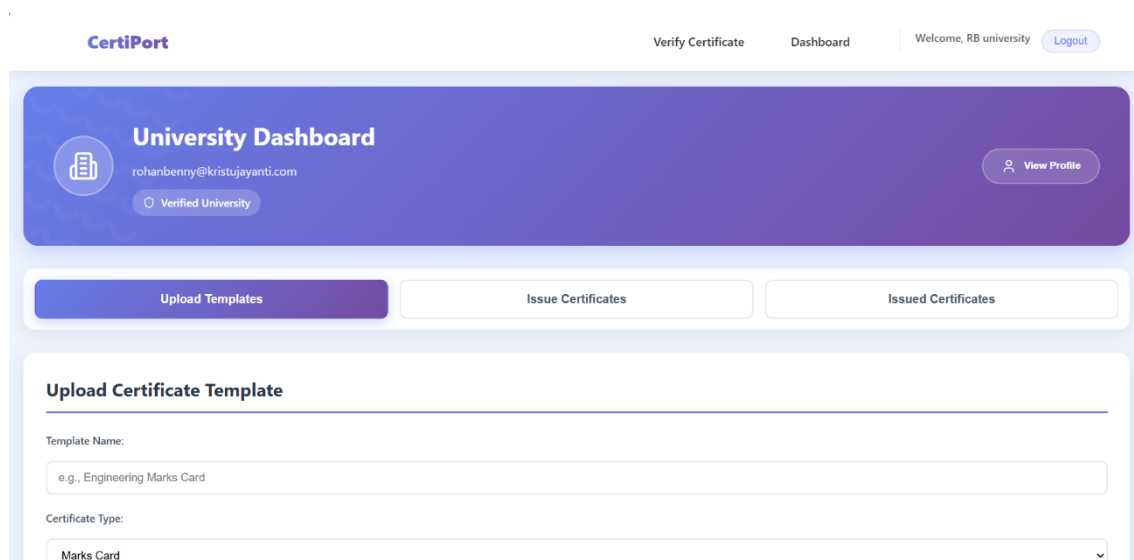


Figure 10.4 University Dashboard

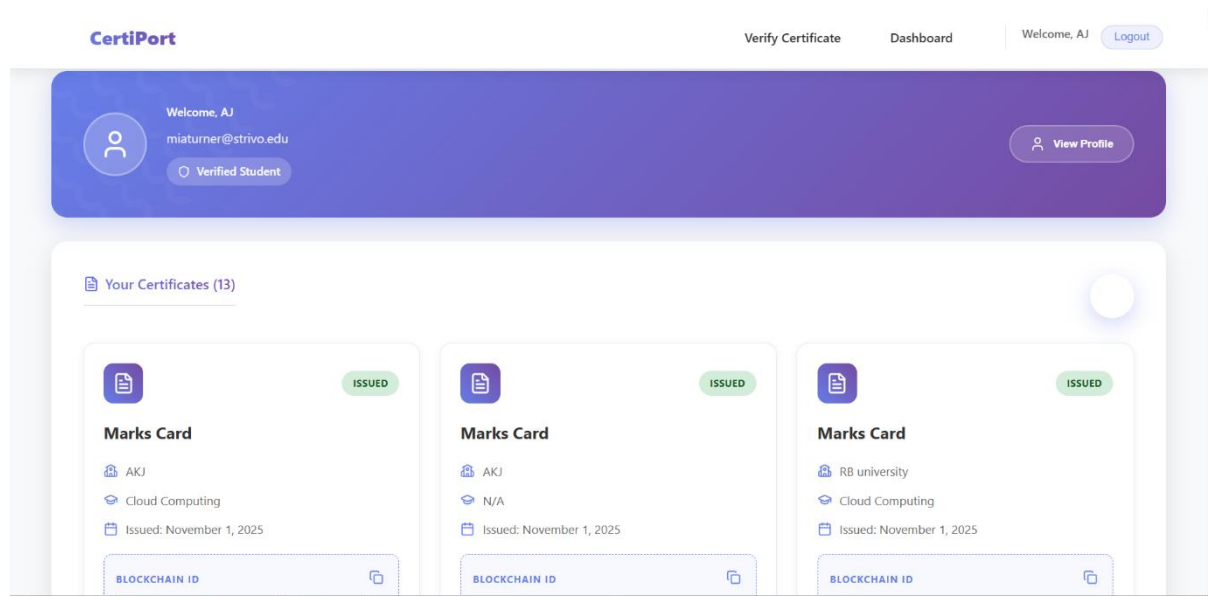


Figure 10.5 Student Dashboard

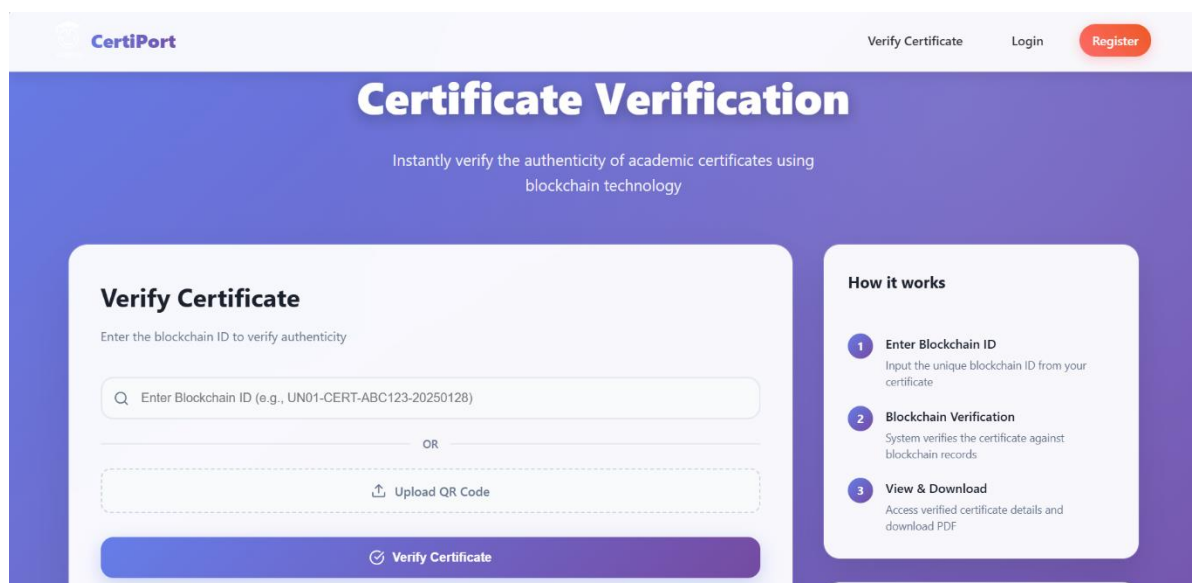


Figure 10.6 Certificate Verification

CertiPort Verify Certificate Dashboard Welcome, RB university Logout

Issued Certificates

Search by ID, Register No, or Name All Types Course dd-mm-yyyy dd-mm-yyyy

Blockchain ID	Student Name	Register No	Type	Course	Issue Date	Status	Actions
un03-CERT-0A1H6W-20251101	Mia	24mcab01	marks card	Cloud Computing	1/11/2025	ISSUED	View Revoke
un03-CERT-FQ6D0M-20251101	pok	24mcab00	migration	MCA	1/11/2025	ISSUED	View Revoke
un03-CERT-LQ2YAG-20251101	jnk	24mcab00	transfer	MCA	1/11/2025	ISSUED	View Revoke
un03-CERT-VJU9RF-20251101	Mia	24mcab01	marks card	Cloud Computing	1/11/2025	ISSUED	View Revoke
un03-CERT-805L8F-20251101	Mia Turner	24mcab01	marks card	Cloud Computing	1/11/2025	ISSUED	View Revoke
un03-CERT-QC18FQ-20251028	jnk	24mcab00	transfer	MCA	28/10/2025	ISSUED	View Revoke

Figure 10.7 Issued Certificates

11. REFERENCES

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Retrieved from <https://bitcoin.org/bitcoin.pdf>
2. Hardhat Documentation. (2024). *Ethereum Development Environment for Professionals*. Retrieved from <https://hardhat.org/>
3. Ether.js Documentation. (2024). *Ethereum JavaScript API*. Retrieved from <https://docs.ethers.io/>
4. React.js Official Documentation. (2024). *A JavaScript Library for Building User Interfaces*. Meta Platforms Inc.
Retrieved from <https://react.dev/>
4. Express.js Documentation. (2024). *Fast, Unopinionated, Minimalist Web Framework for Node.js*.
Retrieved from <https://expressjs.com/>
5. MongoDB Documentation. (2024). *NoSQL Database for Modern Applications*.
Retrieved from <https://www.mongodb.com/docs/>
6. World Economic Forum. (2021). *Blockchain for Education: Lifelong Learning and the Future of Credentialing*.
Retrieved from <https://www.weforum.org/>
7. IBM Blockchain. (2023). *Transforming Education with Digital Credentials*. Dr
Retrieved from <https://www.ibm.com/blockchain/>
8. W3C. (2022). *Verifiable Credentials Data Model 1.1*. World Wide Web Consortium (W3C).
Retrieved from <https://www.w3.org/TR/vc-data-model/>